

SILVER SUITS AI

AI Voice RM Platform

Complete Technical Implementation Guide

Banking Android App & Internet Banking Website

Version	Date	Status	Classification
1.0	May 2026	Engineering Reference	Confidential

Silver Suits AI (Platform Owner) - Builds and operates the Voice RM platform - Ships the voice-rm-sdk to partner banks - Hosts all AI services in Azure India Central - Owns security, compliance, and uptime	Partner Bank (Integration Client) - Integrates the SDK into existing Android app - Embeds SDK into internet banking website - Provides JWT auth, core APIs, DocumentDB - Minimal lift — estimated 2–3 weeks effort
--	---

This document is the single engineering reference for the AI Voice RM platform. Reading it end-to-end gives any developer a complete understanding of every system component, API contract, data model, integration pattern, security control, and delivery timeline — sufficient to build and ship the platform from zero to production.

Primary audience: Silver Suits engineers. Secondary audience: bank integration team (sections marked with 🏦 are relevant to the bank).

Table of Contents

Table of Contents	2
1.1 Product Scope	5
1.2 What Silver Suits Owns vs What the Bank Owns	5
1.3 System at a Glance — Architecture Overview	6
2.1 Repository Boundary.....	6
3.1 APIs the Bank Must Expose to Us.....	7
3.2 Infrastructure the Bank Must Provide.....	8
3.3 Android Integration (Bank Mobile Team) — Effort Estimate.....	9
3.4 Web Integration (Bank Frontend Team) — Effort Estimate.....	9
4.1 Deployment Zones & Network Topology.....	12
4.2 Silver Suits Service Architecture.....	12
4.3 Voice Turn — End-to-End Data Flow.....	12
4.4 AG-UI Form Assistant — Event Flow	13
4.5 Trigger System — Behavioral Rule Flow.....	13
4.6 Failure Modes & Resilience	14
5.1 Monorepo Setup (pnpm workspaces).....	16
5.2 Server Entry Point	16
6.1 Session Management (Redis).....	17
6.2 Manifest (Saga) Structure	18
6.3 Turn State Machine.....	19
6.4 LLM Slot Extraction	20
7.1 AG-UI Router	21
7.2 Agent Runner Pattern	21
8.1 Trigger Service.....	23
8.2 API Gateway Middleware.....	24
9A.1 Bank Adapter (Silver Suits implements this against bank APIs).....	25
9A.2 Speech Proxy with PII Redaction	26
10.1 SDK Component Architecture	29
10.2 Package.json & Build Config.....	29
10.3 VoiceRmProvider (Root Context)	30
10.4 useVoiceSession Hook.....	30
10.5 AG-UI Client (SSE Stream Handler).....	31
10.6 useRageDetect Hook.....	32
10.7 Android Native Bridge Abstraction	33
9.1 Android — VoiceRmActivity.....	36
9.2 Android — VoiceRmBridge (JS Interface).....	37
9.3 Android — Microphone Permission Handler	37
9.4 Android — Security Config.....	38

9.5 Android — AndroidManifest & Deep Link Wiring	38
9.8 Web — NPM SDK Integration	39
9.9 Web — Mounting on a Form Page	39
9.10 Web — CSP Headers Update	40
11.1 Azure Resources Required	42
11.2 Environment Variables — Complete Reference	42
11.3 CI/CD Pipeline	43
11.4 Docker Image	44
12.1 Structured Logging (Pino + PII Redaction)	44
12.2 Observability Stack	45
12.3 Incident Runbooks	45
Redis Down	45
Azure OpenAI Unavailable	46
Bank Core API Down	46
13.1 Token Architecture	48
13.2 P0 Security Controls — Pre Go-Live Checklist	48
13.3 VAPT Scope (Sprint 7–8, Bank-Approved Vendor)	49
14.1 Test Pyramid	51
14.2 Scripted Flow Test — Complete Example	51
14.3 Load Test Specification (k6)	52
15.1 V1 MVP — Weeks 1–8 (Go-Live)	55
15.2 V2 Production — Weeks 9–20	55
15.3 V3 Intelligence — Weeks 21–36	56
15.4 Team Allocation	56
15.5 Demo → Production Migration Checklist	57
15.6 V1 Go-Live Readiness Gate	57
16.1 All Endpoints	60
16.2 POST /api/engine/turn — Complete Contract	60
16.3 POST /api/agui/:agentId — AG-UI Contract	61
16.4 POST /api/trigger Contract	62
16.5 DocumentDB Schema — All Collections	62
16.6 Error Codes Reference	63
16.7 Journey & Agent IDs	63
16.8 Supported Languages	64
16.9 Glossary	64

A

PART A

Product & Integration Overview

§1

What We Are Building

AI voice assistant natively embedded in a partner bank's app and website

1.1 Product Scope

The AI Voice RM (Relationship Manager) platform gives bank customers the ability to speak or type natural-language commands — in any of 10 Indian languages — to complete banking journeys without manually navigating forms. The assistant lives entirely inside the bank's existing application; there is no separate app, no third-party chat widget, and no redirection.

Capability	What the customer experiences	Platform component
Voice-commanded transactions	Say "Rahul ko 500 रुपये भेजो" — the system extracts intent, fills the form, confirms via MPIN, and executes the transfer	Voice Engine + Bank Adapter
AI form assistant	Open IMPS form — agent automatically pre-fills beneficiary, IFSC, and amount from conversation context	AG-UI Service + SDK AguiPanel
Frustration detection & help	After repeated invalid entries, a voice help prompt appears automatically	Trigger Service + useRageDetect
Proactive push nudges	A rule fires after cart abandon — push notification with deep link reopens the journey	Trigger Service + Bank Push Gateway
10 Indian languages	Switch language mid-session; STT, manifests, and TTS all follow	Speech Proxy + manifest i18n
Human RM escalation	If AI cannot resolve, a human RM callback ticket is created silently	Trigger → Bank CRM webhook

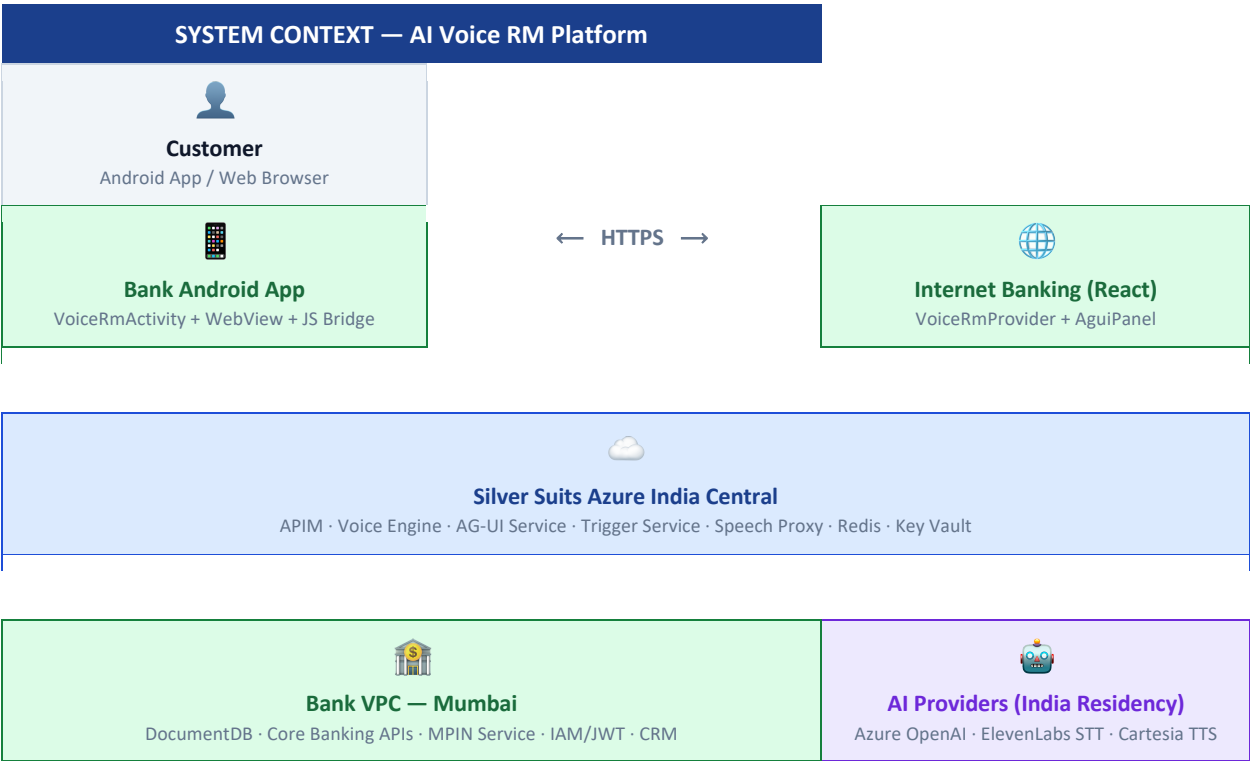
1.2 What Silver Suits Owns vs What the Bank Owns

i The boundary is clear: Silver Suits builds and operates the AI brain. The bank owns the customer, the money, and the app shell. The SDK is the handshake.

Responsibility	Silver Suits	Partner Bank
Voice Engine & AI services	✓ Builds, hosts, scales	— Consumes via API
AG-UI agent runners	✓ Builds, hosts	— Consumes via SSE
Trigger & rule engine	✓ Builds, hosts	— Provides event signals
voice-rm-sdk (npm package)	✓ Builds & publishes	— Installs & integrates
Android WebView shell	— Provides bundle via CDN	✓ Writes VoiceRmActivity + Bridge

Responsibility	Silver Suits	Partner Bank
Internet banking embed	— Provides npm SDK	✓ Imports and wires to forms
Customer JWT issuance	— Validates token	✓ Issues token at login
Core banking APIs	— Calls via adapter interface	✓ Exposes UAT + prod endpoints
DocumentDB (persistence)	— Writes sessions/turns	✓ Hosts cluster, owns data
MPIN verification	— Validates step-up token	✓ Issues step-up token post-PIN
Push notification delivery	— Triggers via bank gateway	✓ Operates FCM/APNs gateway
Security & compliance	✓ VAPT, secret mgmt, PII redact	✓ JWT config, data governance

1.3 System at a Glance — Architecture Overview



§2

Two Codebases, One System

Silver Suits platform repo ↔ Bank integration code — clear boundary

2.1 Repository Boundary

There are exactly two codebases. Silver Suits engineers own Repository 1 end-to-end. The bank's engineers write only Repository 2 — and only need to follow Section 9 of this document to do so.

REPO 1 — voice-rm-platform (Silver Suits)	REPO 2 — Bank Integration Code (Partner Bank) 🏦
<pre>server/ ← all backend services engine/ ← voice engine (sessions, manifests, LLM) agui/ ← AG-UI service (SSE, agents, tools) trigger/ ← trigger service (rules, actuation) speech/ ← STT/TTS proxy adapter/ ← bank core API adapter interface gateway/ ← JWT validation, rate limit middleware packages/ voice-rm-sdk/ ← npm SDK (published to bank) src/ provider/ ← VoiceRmProvider (auth + config) voice/ ← VoiceModal, useVoiceSession agui/ ← AguiPanel, aguiClient, applyStateDelta behavior/ ← useRageDetect, RMHelpPrompt speech/ ← useElevenSpeech, cartesiaTts bridge/ ← nativeBridge (Android postMessage) infra/ ← Terraform (Azure resources) tests/ ← scripted flows, load tests</pre>	<pre># Android (Kotlin) VoiceRmActivity.kt VoiceRmBridge.kt VoiceRmChromeClient.kt network_security_config.xml # Internet Banking (React/JS) App.tsx ← wrap with VoiceRmProvider ImpsTransferPage.tsx ← mount AguiPanel UpiPaymentPage.tsx ← mount VoiceModal # Backend (Bank side) JWT issuer config (RS256 public key) /api/trigger caller (from bank frontend) Core API endpoints (for bank-adapter) DocumentDB schema migration Step-up token service config # That is ALL. ~400 lines total.</pre>

§3

Partner Bank Requirements 🏦

What we need from the bank — concise and specific

i Bank Team: The bank's integration effort is intentionally minimal. We have designed the SDK so the bank writes ~400 lines of integration code. This section tells the bank team exactly what they need to provide.

3.1 APIs the Bank Must Expose to Us

Silver Suits's bank-adapter layer will call these endpoints. The bank must provide UAT versions by Sprint 3 and production versions by Sprint 7.

Endpoint (bank's own API)	Method	Purpose	Required By Sprint
/customers/{cif}/beneficiaries	GET	List saved beneficiaries — returns [{name, account, ifsc, vpa, nickname}]	3
/customers/{cif}/accounts	GET	Account balances — returns [{account_id, type, balance, masked_number}]	3
/payments/upi	POST	Execute UPI payment — must be idempotent on Idempotency-Key header	5
/payments/neft	POST	Execute IMPS/NEFT transfer — idempotent	5
/loans/apply	POST	Submit loan application — idempotent	5
/auth/verify-step-up	POST	Validate step-up JWT issued after MPIN — returns {valid:boolean}	7
/auth/jwks	GET	RS256 public key endpoint for our JWT middleware to validate customer tokens	1
/crm/escalations	POST	Create human RM ticket — called when assign_human_rm trigger fires	6
/push/send	POST	Bank's own FCM/APNs gateway — called when push notification trigger fires (V2)	10

i All bank API calls from Silver Suits will carry the customer's CIF in the URL path, a bank-issued service token in Authorization: Bearer, and an X-Idempotency-Key on payment operations. The bank must validate the service token on their side.

3.2 Infrastructure the Bank Must Provide

Item	Specification	Who Sets Up	When Needed
DocumentDB cluster	MongoDB 5.x compatible, hosted in bank VPC. 7-year retention. Silver Suits writes sessions/turns; bank owns the cluster.	Bank DBA	Sprint 1
DocumentDB connection URI	mongodb+srv:// URI with credentials for Silver Suits write user. Read-only URI for bank analytics.	Bank DBA → Silver Suits DevOps	Sprint 1
Azure Private Link / VPN	Secure tunnel from Silver Suits Azure (India Central) to bank VPC (Mumbai). mTLS client cert rotated 90-day.	Bank Network + Silver Suits DevOps	Sprint 2
RS256 JWT public key (JWKS)	Customer JWTs issued at login. Bank provides JWKS endpoint. Silver Suits validates on every API call.	Bank IAM team	Sprint 1
Service token (for /api/trigger)	Static bearer token Silver Suits issues. Bank frontend/backend uses it to call POST /api/trigger.	Silver Suits → Bank	Sprint 5
Step-up token service	After customer enters MPIN, bank issues a short-lived JWT with action + jti claims.	Bank auth team	Sprint 7

Item	Specification	Who Sets Up	When Needed
	Silver Suits validates it server-side before executing payment.		
Test bank accounts (UAT)	5 test CIF accounts with varied balances, beneficiaries for full scripted flow testing	Bank QA team	Sprint 3

3.3 Android Integration (Bank Mobile Team) — Effort Estimate

✓ **Bank Mobile Team:** Estimated effort: 1 Android developer × 4–5 weeks. All code is provided in §9 of this document with copy-paste Kotlin implementations.

Task	Effort	Section Reference
Create VoiceRmActivity with hardened WebView settings	2 days	§9.1
Implement VoiceRmBridge (4 methods: getAuthToken, navigateTo, verifyMpin, logEvent)	2 days	§9.2
Implement VoiceRmChromeClient for microphone permission handling	1 day	§9.3
Security config: network_security_config.xml, URL allowlist, disable file access	1 day	§9.4
Wire VoiceRmActivity into existing app navigation (deep links, FAB button)	2 days	§9.5
Test with staging SDK bundle; verify bridge callbacks, mic, navigation	3 days	§9.6
MPIN activity integration (calls existing MpinActivity, returns result to WebView)	2 days	§9.2
Production CDN URL switch, final security review	1 day	§9.7

3.4 Web Integration (Bank Frontend Team) — Effort Estimate

✓ **Bank Frontend Team:** Estimated effort: 1 frontend developer × 2–3 weeks. Install our npm package, wrap the app with VoiceRmProvider, and mount AguiPanel on each form page.

Task	Effort	Section Reference
npm install @silversuits/voice-rm-sdk; configure VoiceRmProvider in App.tsx	1 day	§9.8
Mount AguiPanel on each target form page (IMPS, UPI, Loan, FD — 4 forms)	3 days	§9.9
Wire useRageDetect on high-friction form pages (IMPS, UPI minimum)	1 day	§9.10

Task	Effort	Section Reference
Integrate bankApi.postTrigger() call on frustration event	1 day	§9.10
Test all journeys in staging; verify SSE stream, form fill, state delta	2 days	§9.11
Production env config; CSP headers update for Silver Suits API domain	1 day	§9.12

B

PART B

System Architecture — HLD

§4

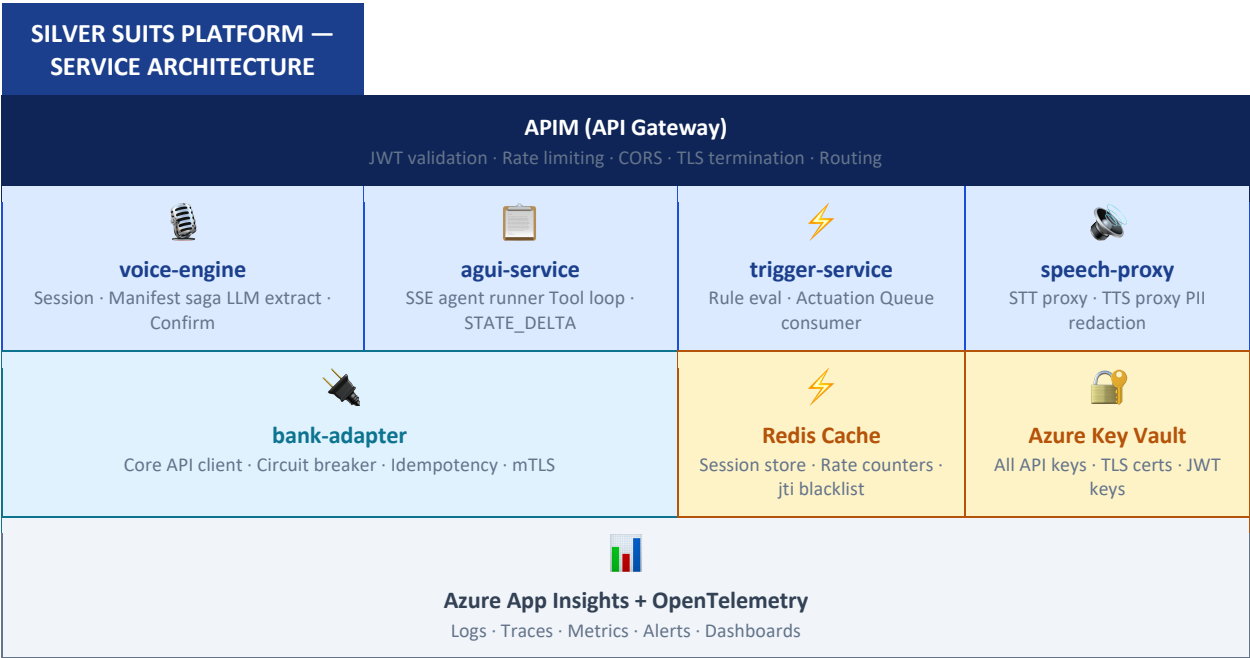
High-Level Design

Deployment topology · Service boundaries · Network zones · Data flows

4.1 Deployment Zones & Network Topology

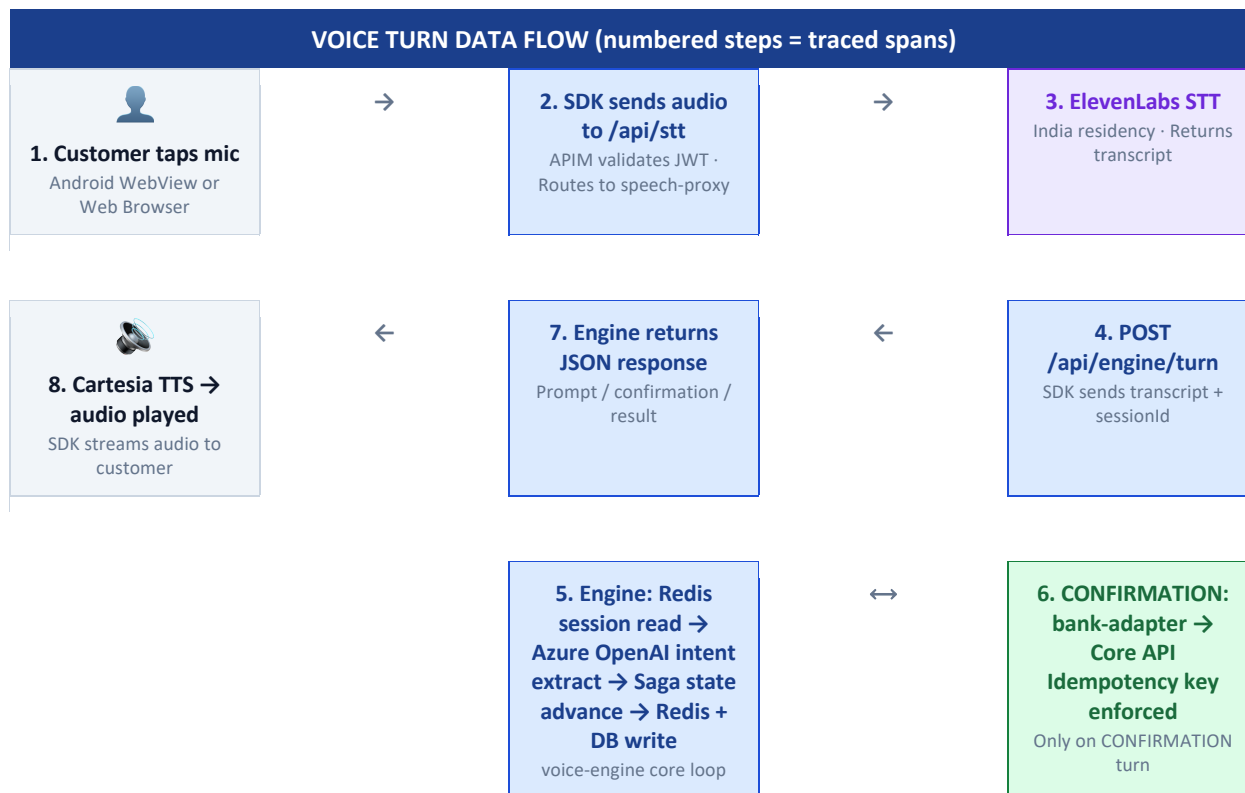
Zone	Components	Connects To	Protocol
Bank DMZ — Customer Device	Android app (WebView shell), IB website (React + SDK)	Silver Suits APIM	HTTPS/443 only
Silver Suits Azure — India Central	APIM, App Service (voice-engine, agui, trigger, speech-proxy), Redis Cache, Key Vault, Container Registry, App Insights	Bank VPC + AI providers	Private Link (mTLS), HTTPS outbound
Bank VPC — Mumbai	DocumentDB cluster, Core Banking REST APIs, MPIN service, IAM/JWKS, CRM webhook, Push gateway	Silver Suits Azure	Azure Private Link (mTLS)
AI Providers — India Residency	Azure OpenAI (gpt-4o, East India), ElevenLabs Scribe v1, Cartesia Sonic-2	Silver Suits Azure	HTTPS outbound from App Service

4.2 Silver Suits Service Architecture

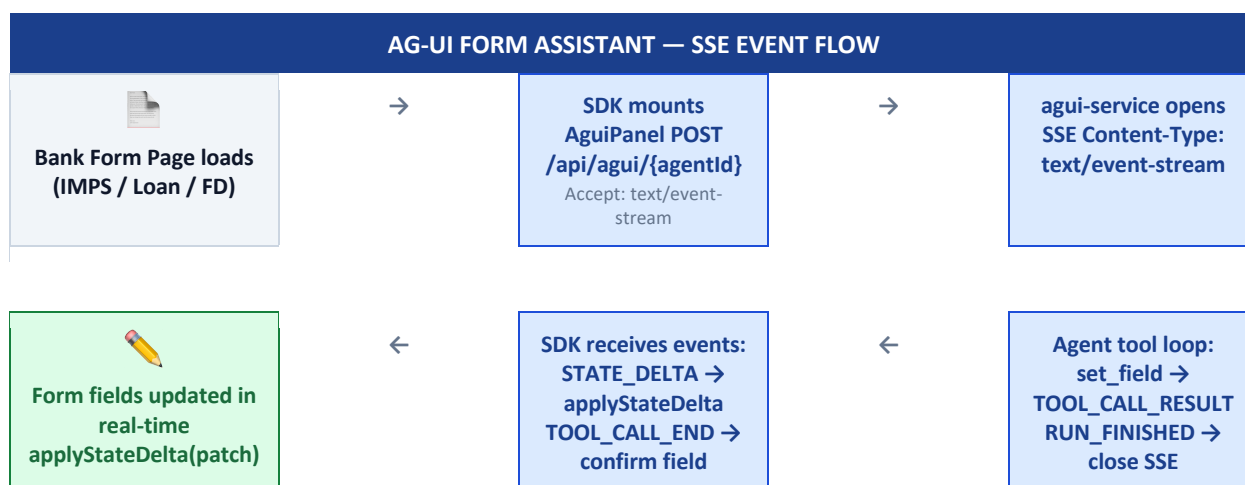


4.3 Voice Turn — End-to-End Data Flow

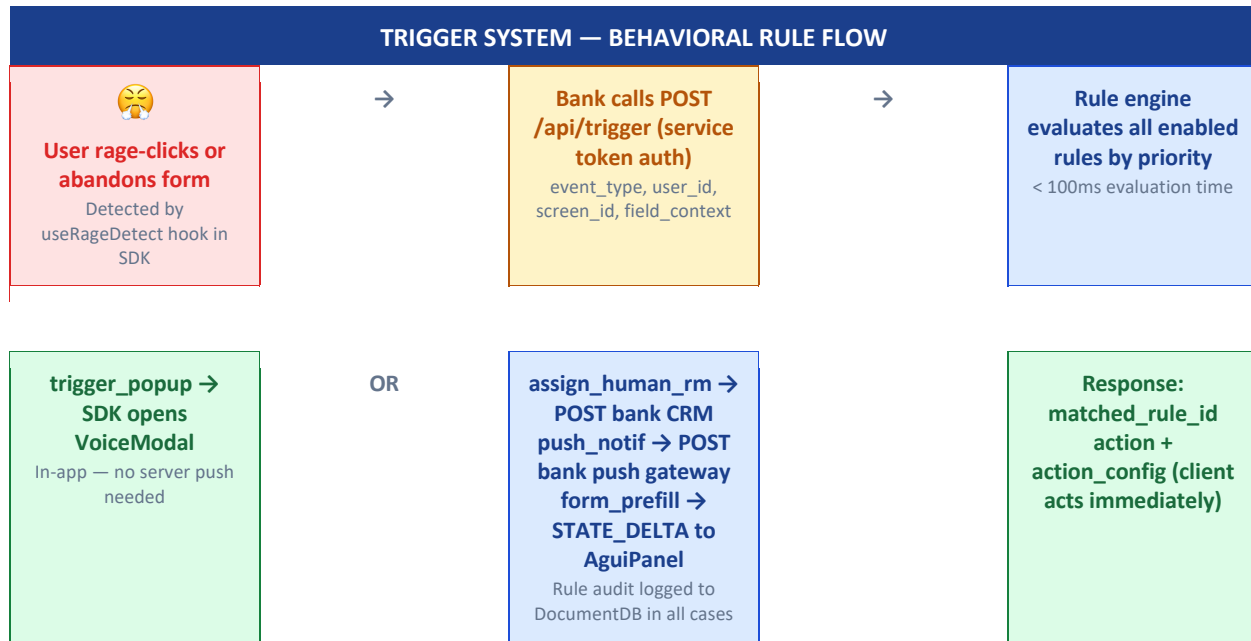
i Every numbered step below is a traced span in OpenTelemetry. The total p95 target is < 1.5 seconds from mic-tap to audio response.



4.4 AG-UI Form Assistant — Event Flow



4.5 Trigger System — Behavioral Rule Flow



4.6 Failure Modes & Resilience

Failure	Detection	Recovery Strategy	Customer Impact
Redis unavailable	Connection error → App Insights alert	Fail-open: in-process Map for ≤60s; circuit breaker; DevOps paged	Degraded: single-pod only for 60s
Azure OpenAI timeout (>3s)	OTel span: LLM duration	Retry once after 500ms. If second fails, return "could not understand" + escalation trigger fires	Sees retry message; offered human RM
ElevenLabs STT down	speech-proxy 5xx counter	Return STT_UNAVAILABLE error; SDK shows "Voice unavailable — please type" fallback UI	Types instead of speaks
Bank Core API 5xx	bank-adapter circuit breaker	Opens after 3 consecutive failures. Returns FAILED state. Idempotency key retained for retry.	Sees failure; can retry when bank recovers
SSE connection dropped	Client EventSource onerror	SDK reconnects with same thread_id; server emits STATE_SNAPSHOT on reconnect (V2)	Brief reconnect flash; form state restored
APIM rate limited (429)	APIM throttle counter	SDK exponential backoff + shows "please wait" message. Trigger fires push for re-engagement.	Brief wait message

C

PART C

Silver Suits Platform — Complete Build Guide

§5

Repository Setup & Monorepo Structure

How to initialize and structure the codebase from day one

5.1 Monorepo Setup (pnpm workspaces)

MONOREPO STRUCTURE

```
# One-time setup - run in repo root
node --version # must be 20.x LTS
npm install -g pnpm

pnpm init
# Edit package.json to add workspaces:
# { "workspaces": ["packages/*", "server"] }

# Install workspace deps
pnpm install

# Directory structure
voice-rm-platform/
├── server/                                ← Express API server (all backend services)
│   ├── src/
│   │   ├── index.js                    ← entry point, registers all routes
│   │   ├── gateway/                   ← JWT middleware, rate limit, CORS
│   │   ├── engine/                    ← voice engine (sessions, manifests, LLM)
│   │   ├── agui/                      ← AG-UI service (SSE, agents)
│   │   ├── trigger/                  ← trigger service (rules, actuation)
│   │   ├── speech/                   ← STT/TTS proxies
│   │   ├── adapter/                  ← bank adapter (interface + circuit breaker)
│   │   └── lib/                      ← shared: logger, redis, db, errors
│   ├── manifests/                    ← journey definitions (JSON)
│   ├── tests/                        ← scripted flows, unit tests
│   └── package.json
├── packages/
│   └── voice-rm-sdk/                 ← TypeScript SDK (shipped to banks)
│       ├── src/
│       └── package.json
├── infra/                            ← Terraform (Azure resources)
├── .github/workflows/                ← CI/CD
└── package.json                      ← root workspace config
```

5.2 Server Entry Point

server/src/index.js

```
// server/src/index.js
const express = require('express');
const { createClient } = require('redis');
const { connectDb } = require('./lib/db');
const { loadSecrets } = require('./lib/secrets'); // Azure Key Vault
const gatewayMiddleware = require('./gateway');
const engineRouter = require('./engine/router');
const aguiRouter = require('./agui/router');
```



```

const triggerRouter = require('./trigger/router');
const speechRouter = require('./speech/router');

async function start() {
  await loadSecrets();           // Pull all secrets from Key Vault at startup
  await connectDb();             // DocumentDB connection

  const app = express();
  app.use(express.json({ limit: '1mb' }));
  app.use(gatewayMiddleware);    // CORS, JWT, rate limit on all routes

  app.get('/api/health', (req, res) => res.json({ status: 'ok', version:
process.env.APP_VERSION }));
  app.use('/api/engine', engineRouter);
  app.use('/api/agui', aguiRouter);
  app.use('/api/trigger', triggerRouter);
  app.use('/api', speechRouter); // /api/stt, /api/tts

  const PORT = process.env.PORT || 3000;
  app.listen(PORT, () => logger.info({ port: PORT }, 'voice-rm-server started'));
}
start().catch(err => { logger.error(err); process.exit(1); });

```

§6

Voice Engine — Complete Implementation

Sessions · Manifest sagas · LLM intent extraction · Turn state machine

6.1 Session Management (Redis)

server/src/engine/session.js

```

// server/src/engine/session.js
const redis = require('../lib/redis');
const db = require('../lib/db');
const { v4: uuid } = require('uuid');

const SESSION_TTL = 30 * 60; // 30 minutes

async function createSession(userId, lang) {
  const sessionId = `sess_${uuid()}`;
  const session = { sessionId, userId, lang, state: "IDLE", action: null,
    slots: {}, history: [], turnCount: 0, createdAt: Date.now() };
  await redis.set(`session:${sessionId}`, JSON.stringify(session), { EX: SESSION_TTL });
  await db.sessions.insertOne({ sessionId, userId, lang, outcome: null,
    startedAt: new Date(), platform: "web" });
  return session;
}

async function getSession(sessionId) {
  const raw = await redis.get(`session:${sessionId}`);
  if (!raw) throw Object.assign(new Error("Session not found"), { code:
"SESSION_EXPIRED" });
  return JSON.parse(raw);
}

```

```

async function saveSession(session) {
  await redis.set(`session:${session.sessionId}`, JSON.stringify(session), { EX:
SESSION_TTL });
}

async function endSession(session, outcome) {
  await redis.del(`session:${session.sessionId}`);
  await db.sessions.updateOne({ sessionId: session.sessionId },
    { $set: { outcome, endedAt: new Date(), turnCount: session.turnCount } });
}

module.exports = { createSession, getSession, saveSession, endSession };

```

6.2 Manifest (Saga) Structure

Each banking journey is defined as a manifest — a declarative JSON configuration that tells the engine: what slots to collect, how to prompt for each, how to confirm, and what bank adapter method to call.

server/manifests/sendMoney.json

```

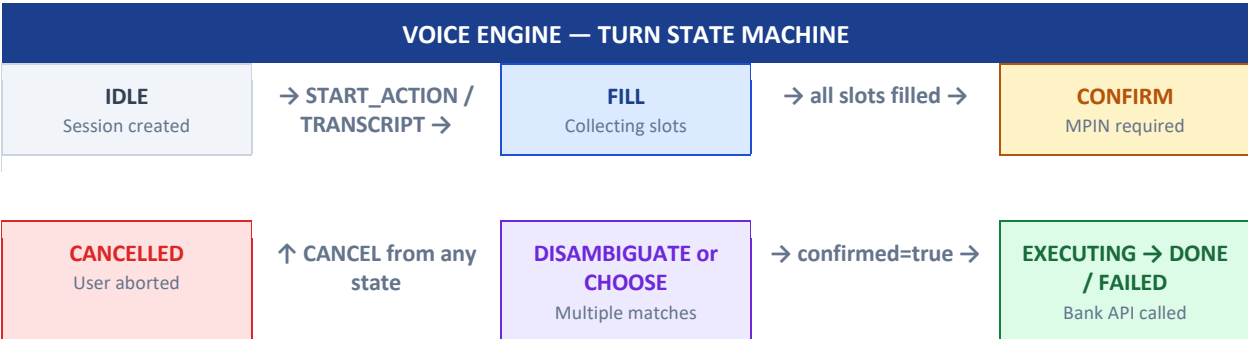
// server/manifests/sendMoney.json
{
  "id": "send_money",
  "displayName": { "en": "Send Money", "hi": "पैसे भेजें" },
  "slots": [
    {
      "id": "beneficiary",
      "kind": "contact_lookup",
      "required": true,
      "prompt": { "en": "Who would you like to send money to?", "hi": "किससे पैसे भेजने हैं?" },
    },
    {
      "id": "amount",
      "kind": "currency_inr",
      "required": true,
      "prompt": { "en": "How much would you like to send?", "hi": "कितना भेजना है?" },
      "validation": { "min": 1, "max": 200000 }
    },
    {
      "id": "account_type",
      "kind": "single_choice",
      "required": false,
      "defaultValue": "primary",
      "prompt": { "en": "Which account to debit?", "hi": "कौन सा खाता?" },
      "options": "dynamic:bankAdapter.getAccounts"
    }
  ],
  "confirmationTemplate": {
    "en": "Send ₹{amount} to {beneficiary.name}? Please verify MPIN.",
    "hi": "₹{amount} {beneficiary.name} को भेजें? MPIN दर्ज करें!"
  },
  "execute": "bankAdapter.sendUpi",
  "successTemplate": { "en": "₹{amount} sent to {beneficiary.name}. Ref: {txn_ref}",
    "hi": "..." },
  "failureTemplate": { "en": "Transfer failed. Please try again.", "hi": "..." }
}

```

```
}

```

6.3 Turn State Machine



```
server/src/engine/stateMachine.js

// server/src/engine/stateMachine.js - core turn handler
async function processTurn(session, input, userId) {
  const { type } = input;

  if (type === 'CANCEL') return transition(session, 'CANCELLED');
  if (type === 'SET_LANG') { session.lang = input.lang; return session; }

  if (type === 'INIT') {
    session.history.push({ role:"assistant", text: await greet(session.lang) });
    return session;
  }

  if (type === 'START ACTION') {
    session.action = input.action;
    session.manifest = await loadManifest(input.action);
    session.state = 'FILL';
    session.pending = nextSlot(session.manifest, session.slots);
    return session;
  }

  if (type === 'TRANSCRIPT') {
    // 1. Extract intent + slots from utterance via LLM
    const extracted = await extractSlots(input.text, session.manifest, session.lang, session.slots);
    Object.assign(session.slots, extracted);
    session.history.push({ role:"user", text: input.text });

    // 2. Check if all required slots are filled
    const next = nextSlot(session.manifest, session.slots);
    if (!next) {
      session.state = 'CONFIRM';
      session.pending = buildConfirmation(session.manifest, session.slots, session.lang);
    } else {
      session.pending = next;
    }
    return session;
  }

  if (type === 'CONFIRMATION' && input.confirmed) {

```

```

    // Validate step-up token (bank MPIN service)
    await bankAdapter.verifyStepUpToken(userId, input.stepUpToken, session.action);
    session.state = 'EXECUTING';
    const result = await bankAdapter[session.manifest.execute](userId, session.slots,
input.idempotencyKey);
    session.state = result.ok ? 'DONE' : 'FAILED';
    session.result = result;
    await endSession(session, session.state);
    return session;
  }
}

```

6.4 LLM Slot Extraction

server/src/engine/llm.js

```

// server/src/engine/llm.js
const { AzureOpenAI } = require('openai');
const client = new AzureOpenAI({ endpoint: process.env.AZURE_OPENAI_ENDPOINT });

async function extractSlots(utterance, manifest, lang, existingSlots) {
  const slotDefs = manifest.slots.map(s =>
    `${s.id} (${s.kind}): ${s.extractionHint || ""}`
  ).join("\n");

  const prompt = [
    { role: "system", content: `You are a banking voice assistant. Extract slot values
from the user utterance.
Language: ${lang}. Existing slots already collected:
${JSON.stringify(existingSlots)}.
Required slots:\n${slotDefs}.
Respond ONLY with a JSON object of { slotId: extractedValue } for slots found in
the utterance.
If a slot value is not found, omit it. Never invent values.` },
    { role: "user", content: utterance }
  ];

  const resp = await client.chat.completions.create({
    model: 'gpt-4o',
    messages: prompt,
    response_format: { type: 'json_object' },
    max_tokens: 200,
    temperature: 0, // deterministic extraction
  });

  return JSON.parse(resp.choices[0].message.content);
}

module.exports = { extractSlots };

```

§7

AG-UI Service — Complete Implementation

SSE server · Agent runners · Tool loop · Event types

7.1 AG-UI Router

server/src/agui/router.js

```
// server/src/agui/router.js
const express = require('express');
const { verifyJwt, rateLimitAgui } = require('../gateway');
const agentRegistry = require('./agentRegistry');
const { writeTurnDb } = require('../lib/db');
const router = express.Router();

router.post('/:agentId', verifyJwt, rateLimitAgui, async (req, res) => {
  const { agentId } = req.params;
  const { thread_id, run_id, state, messages, context, forwarded_props } = req.body;
  const userId = req.user.sub;

  // — SSE headers (all four required) —————
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');
  res.setHeader('X-Accel-Buffering', 'no'); // disable Nginx buffering
  res.flushHeaders();

  const emit = (type, data) => {
    res.write(`event: ${type}\n`);
    res.write(`data: ${JSON.stringify(data)}\n\n`);
  };

  emit("RUN_STARTED", { thread_id, run_id });

  try {
    const agent = agentRegistry.get(agentId);
    if (!agent) throw Object.assign(new Error("Unknown agent"), { code:
"AGENT_NOT_FOUND" });
    await agent.run({ state, messages, context, forwarded_props, emit, userId });
    emit("RUN_FINISHED", { thread_id, run_id });
  } catch (err) {
    emit("RUN_ERROR", { message: err.message, code: err.code ?? "AGENT_ERROR" });
  } finally {
    res.end();
  }
});

module.exports = router;
```

7.2 Agent Runner Pattern

server/src/agui/agents/impsTransferAgent.js

```
// server/src/agui/agents/impsTransferAgent.js
const { AzureOpenAI } = require('openai');
const client = new AzureOpenAI({ endpoint: process.env.AZURE_OPENAI_ENDPOINT });

const TOOLS = [
  {
    type: "function",
    function: {
      name: "set_field",
      description: "Set a form field value based on user input",
      parameters: {
        type: "object",
        properties: {
```

```

        field id: { type: "string", enum:
["beneficiary_name","account_number","ifsc_code","amount","remarks"] },
        value:      { type: "string", description: "The extracted value to set" },
    },
    required: ["field_id", "value"]
}
}
}
];

async function run({ state, messages, context, forwarded_props, emit, userId }) {
    const userProfile = context.find(c => c.type === "user_profile")?.data || {};
    const llmMessages = [
        { role: "system", content: buildSystemPrompt(userProfile) },
        ...messages
    ];

    const currentState = { ...state };
    let continueLoop = true;

    while (continueLoop) {
        const resp = await client.chat.completions.create({
            model: 'gpt-4o',
            messages: llmMessages,
            tools: TOOLS,
            tool_choice: 'auto',
            stream: false, // tool loop does not stream text
        });

        const msg = resp.choices[0].message;
        if (msg.tool_calls?.length) {
            for (const tc of msg.tool_calls) {
                emit("TOOL_CALL_START", { tool_call_id: tc.id, tool_name: tc.function.name });
                const args = JSON.parse(tc.function.arguments);
                emit("TOOL_CALL_ARGS", { tool_call_id: tc.id, args });
                emit("TOOL_CALL_END", { tool_call_id: tc.id });

                // Apply tool result
                if (tc.function.name === "set_field") {
                    currentState[args.field_id] = args.value;
                    const patch = [{ op: "replace", path: `/${args.field_id}`, value: args.value
                }];

                emit("STATE_DELTA", { delta: patch });
                emit("TOOL_CALL_RESULT", { tool_call_id: tc.id, result: { success: true } });
            }
            llmMessages.push({ role: "tool", content: JSON.stringify({ success: true }),
tool_call_id: tc.id });
        }
        llmMessages.push(msg);
    } else { continueLoop = false; }

    if (resp.choices[0].finish_reason === 'stop' && !msg.tool_calls?.length)
        continueLoop = false;
    }
}

module.exports = { run };

```

8.1 Trigger Service

server/src/trigger/router.js

```
// server/src/trigger/router.js
const express = require('express');
const { verifyServiceToken } = require('../gateway');
const { evaluateRules } = require('../ruleEngine');
const { actuate } = require('../actuationHandlers');
const db = require('../lib/db');
const router = express.Router();

router.post('/', verifyServiceToken, async (req, res) => {
  const { event_type, user_id, screen_id, field_context, lang, device } = req.body;

  // Evaluate rules against this event
  const match = await evaluateRules({ event_type, user_id, screen_id, field_context });

  // Write audit regardless of match
  await db.ruleAudit.insertOne({
    ruleId: match?.ruleId || null,
    userId: user_id, event_type, screen_id, action: match?.action || "none", ts: new
    Date()
  });

  if (!match) return res.json({ matched_rule_id: null, action: "none" });

  // Actuate side-effects that require server action (e.g. assign_human_rm)
  await actuate(match, { user_id, lang, device });

  res.json({ matched_rule_id: match.ruleId, action: match.action, action_config:
  match.action_config });
});

module.exports = router;
```

server/src/trigger/ruleEngine.js

```
// server/src/trigger/ruleEngine.js
const db = require('../lib/db');
const redis = require('../lib/redis');

let rulesCache = null;
let cacheLoadedAt = 0;
const CACHE_TTL_MS = 60_000; // 60 seconds - ops can override rules; takes effect
within 1 min

async function loadRules() {
  if (Date.now() - cacheLoadedAt < CACHE_TTL_MS && rulesCache) return rulesCache;
  rulesCache = await db.configRules.find({ enabled: true }).sort({ priority: -1
  }).toArray();
  cacheLoadedAt = Date.now();
  return rulesCache;
}

async function evaluateRules(event) {
  const rules = await loadRules();
  for (const rule of rules) {
    if (!matchesConditions(rule.conditions, event)) continue;
```

```

    // Check cooldown — prevent re-triggering same rule for same user within cooldown
    window
    const coolKey = `cooldown:${rule._id}:${event.user_id}`;
    const inCooldown = await redis.get(coolKey);
    if (inCooldown) continue;
    await redis.set(coolKey, "1", { EX: rule.cooldown_seconds || 300 });
    return { ruleId: rule._id, action: rule.action, action_config: rule.action_config };
  };
}
return null;
}

function matchesConditions(conditions, event) {
  return Object.entries(conditions).every(([path, expected]) => {
    const actual = path.split(".").reduce((o, k) => o?.[k], event);
    if (typeof expected === "object" && expected.$gte !== undefined) return actual >=
    expected.$gte;
    return actual === expected;
  });
}

module.exports = { evaluateRules };

```

8.2 API Gateway Middleware

server/src/gateway/index.js

```

// server/src/gateway/index.js
const jwt = require('jsonwebtoken');
const jwksClient = require('jwks-rsa');
const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis').default;
const redis = require('../lib/redis');
const { piiRedact } = require('../lib/log');

// — JWKS client (validates bank-issued customer JWTs) —————
const jwks = jwksClient({ jwksUri: process.env.BANK_JWKS_URI, cache: true, cacheMaxAge:
600_000 });

async function verifyJwt(req, res, next) {
  const token = req.headers.authorization?.slice(7);
  if (!token) return res.status(401).json({ error: 'UNAUTHORIZED' });
  try {
    const { kid } = jwt.decode(token, { complete: true }).header;
    const key = await jwks.getSigningKey(kid);
    req.user = jwt.verify(token, key.getPublicKey(), { algorithms: ["RS256"],
      audience: process.env.JWT_AUDIENCE, issuer:
process.env.JWT_ISSUER });
    next();
  } catch (e) { res.status(401).json({ error: 'INVALID_TOKEN', detail: e.message }); }
}

// — Service token for /api/trigger (bank backend calls this) —————
function verifyServiceToken(req, res, next) {
  const token = req.headers.authorization?.slice(7);
  if (token !== process.env.TRIGGER_SERVICE_TOKEN)
    return res.status(401).json({ error: 'UNAUTHORIZED' });
  next();
}

// — Rate limiters (per user, stored in Redis) —————
const engineLimiter = rateLimit({ windowMs: 60*60*1000, max: 60,

```



```

    keyGenerator: r => r.user?.sub || r.ip,
    store: new RedisStore({ sendCommand: (...a) => redis.sendCommand(a) }),
    message: { error: 'RATE_LIMITED' } });

const aguiliLimiter = rateLimit({ windowMs: 60*60*1000, max: 20,
  keyGenerator: r => r.user?.sub || r.ip,
  store: new RedisStore({ sendCommand: (...a) => redis.sendCommand(a) }),
  message: { error: 'RATE_LIMITED' } });

// — CORS (bank domains only) —————
const ALLOWED = (process.env.ALLOWED_ORIGINS || '').split(',').map(s => s.trim());
function corsMiddleware(req, res, next) {
  const origin = req.headers.origin;
  if (origin && ALLOWED.includes(origin)) {
    res.setHeader('Access-Control-Allow-Origin', origin);
    res.setHeader('Access-Control-Allow-Headers', 'Authorization, Content-Type,
Accept');
    res.setHeader('Access-Control-Allow-Methods', 'POST, GET, OPTIONS');
  }
  if (req.method === 'OPTIONS') return res.sendStatus(204);
  next();
}

module.exports = { verifyJwt, verifyServiceToken, engineLimiter, aguiliLimiter,
corsMiddleware };

```

§9 (Silver Suits)

Bank Adapter & Speech Proxy

Core API adapter interface · Circuit breaker · STT/TTS proxy · PII redaction

9A.1 Bank Adapter (Silver Suits implements this against bank APIs)

server/src/adapter/bankAdapter.js

```

// server/src/adapter/bankAdapter.js
const axios = require('axios');
const CircuitBreaker = require('opossum');

// All bank API calls go through this client with mTLS
const bankAxios = axios.create({
  baseURL: process.env.BANK_API_BASE,
  httpsAgent: new (require("https").Agent)({
    cert: fs.readFileSync(process.env.MTLS_CERT_PATH),
    key: fs.readFileSync(process.env.MTLS_KEY_PATH),
  }),
  headers: { Authorization: `Bearer ${process.env.BANK_SERVICE_TOKEN}` }
});

// Circuit breaker wraps every bank API call
function withBreaker(fn, name) {
  const breaker = new CircuitBreaker(fn, {
    timeout: 5000, // fail after 5s
    errorThresholdPercentage: 50, // open after 50% failures
    resetTimeout: 30000, // retry after 30s
    volumeThreshold: 3, // need at least 3 calls to track
  });
}

```

```

    breaker.on("open",    () => logger.warn({ name }, "circuit open"));
    breaker.on("close",   () => logger.info({ name }, "circuit closed"));
    return breaker;
  }

  const searchContactsBreaker = withBreaker(async (userId, query) => {
    const { data } = await bankAxios.get(`/customers/${userId}/beneficiaries`, { params:
    { q: query } });
    return data;
  }, "searchContacts");

  const sendUpiBreaker = withBreaker(async (userId, payload, idempotencyKey) => {
    const { data } = await bankAxios.post("/payments/upi", payload, {
      headers: { "X-Idempotency-Key": idempotencyKey, "X-User-Id": userId }
    });
    return data;
  }, "sendUpi");

  module.exports = {
    searchContacts: (uid, q) => searchContactsBreaker.fire(uid, q),
    getAccounts:    (uid)    => withBreaker(async () => {
      const {data} = await bankAxios.get(`/customers/${uid}/accounts`);
      return data; }, "getAccounts").fire(),
    sendUpi:       (uid, pl, key) => sendUpiBreaker.fire(uid, pl, key),
    internalTransfer: (uid, pl, key) => withBreaker(async () => {
      const {data} = await bankAxios.post("/payments/neft", pl, {
        headers: { "X-Idempotency-Key": key } }); return data; },
    "neft").fire(),
    verifyStepUpToken: (uid, token, action) => withBreaker(async () => {
      const {data} = await bankAxios.post("/auth/verify-step-up", {token,
      action, userId: uid});
      if (!data.valid) throw Object.assign(new Error("Invalid step-up"),
      {code:"INVALID_STEP_UP"});
      return data; }, "stepUp").fire(),
  };

```

9A.2 Speech Proxy with PII Redaction

server/src/speech/router.js

```

// server/src/speech/router.js
const express = require('express');
const { verifyJwt } = require('../gateway');
const { piiRedact } = require('../lib/log');
const router = express.Router();

// — STT — ElevenLabs Scribe —————
router.post('/stt', verifyJwt, async (req, res) => {
  const audioBuffer = req.body; // raw audio from client
  const lang = req.headers["x-lang"] || "hi";

  const form = new FormData();
  form.append("audio", new Blob([audioBuffer], { type: "audio/webm" }));
  form.append("language_code", lang);
  form.append("model_id", "scribe_v1");

  const resp = await fetch("https://api.elevenlabs.io/v1/speech-to-text", {
    method: "POST",
    headers: { "xi-api-key": process.env.ELEVENLABS_API_KEY },
    body: form
  });

```

```
const data = await resp.json();
const rawText = data.text || "";
// PII redact BEFORE logging (never log raw audio or raw transcript)
const safeText = piiRedact(rawText);
logger.info({ event: "stt_complete", lang, chars: safeText.length });

res.json({ text: rawText }); // return original to client; only logs are redacted
});

// — TTS — Cartesia Sonic-2 —————
router.post('/tts', verifyJwt, async (req, res) => {
  const { text, lang, voice_id } = req.body;

  const resp = await fetch("https://api.cartesia.ai/tts/bytes", {
    method: "POST",
    headers: { "X-API-Key": process.env.CARTESIA_API_KEY, "Content-Type":
"application/json" },
    body: JSON.stringify({ transcript: text, model_id: "sonic-2", voice: { mode:"id",
id: voice_id || "indian-en-female" },
                        language: lang, output_format: { container:"mp3",
sample_rate:44100 } })
  });

  res.setHeader("Content-Type", "audio/mpeg");
  resp.body.pipeTo(new WritableStream({ write(chunk) { res.write(chunk); }, close() {
res.end(); } }));
});

module.exports = router;
```

D

PART D

SDK Build — @silversuits/voice-rm-sdk

§10

SDK Architecture & Build

Package structure · Components · Hooks · Android bridge · SSE client

10.1 SDK Component Architecture

SDK COMPONENT ARCHITECTURE — @silversuits/voice-rm-sdk			
VoiceRmProvider			
React context: apiBase, getToken(), lang, onError			
 VoiceModal Mic UI · Recording state useVoiceSession hook	 AguiPanel AG-UI form assistant SSE event rendering	 useRageDetect Click frustration hook markInvalidField()	 RMHelpPrompt Proactive help bubble Automatic display
useVoiceSession → engineClient.ts POST /api/engine/turn · Session state · sendUtterance()	aguiClient.ts → SSE stream runAgent() · EventSource · applyStateDelta()	useElevenSpeech Mic capture → /api/stt	cartesiaTts.ts /api/tts → audio playback
nativeBridge.ts — Android WebView postMessage adapter getAuthToken · navigateTo · verifyMpin · logEvent — abstracts __bankBridge JS interface			

10.2 Package.json & Build Config

```
packages/voice-rm-sdk/package.json + rollup.config.js

// packages/voice-rm-sdk/package.json
{
  "name": "@silversuits/voice-rm-sdk",
  "version": "1.0.0",
  "description": "AI Voice RM SDK for partner banks",
  "main": "dist/index.cjs.js",
  "module": "dist/index.esm.js",
  "types": "dist/index.d.ts",
  "exports": {
    ".": {
      "import": "./dist/index.esm.js",
      "require": "./dist/index.cjs.js",
      "types": "./dist/index.d.ts"
    }
  },
  "peerDependencies": { "react": ">=18.0.0", "react-dom": ">=18.0.0" },
  "devDependencies": { "rollup": "^4", "typescript": "^5", "@rollup/plugin-typescript":
    "^11" },
  "scripts": {
    "build": "rollup -c",
  }
}
```

```

    "build:watch": "rollup -c --watch",
    "typecheck": "tsc --noEmit"
  },
  "files": ["dist/", "src/"]
}

// rollup.config.js
import typescript from '@rollup/plugin-typescript';
import resolve from '@rollup/plugin-node-resolve';
import commonjs from '@rollup/plugin-commonjs';
export default {
  input: 'src/index.ts',
  external: ['react', 'react-dom'], // Never bundle React
  output: [
    { file: 'dist/index.esm.js', format: 'esm', sourcemap: true },
    { file: 'dist/index.cjs.js', format: 'cjs', sourcemap: true },
  ],
  plugins: [typescript({ declaration: true, declarationDir: "dist" }), resolve(), commonjs()],
};

```

10.3 VoiceRmProvider (Root Context)

src/provider/VoiceRmProvider.tsx

```

// packages/voice-rm-sdk/src/provider/VoiceRmProvider.tsx
import React, { createContext, useContext, ReactNode } from 'react';

interface VoiceRmConfig {
  apiBase: string;
  getToken: () => string | Promise<string>; // Called fresh per API request
  lang?: string;
  onError?: (err: Error) => void;
  onSessionEnd?: (outcome: SessionOutcome) => void;
}

const VoiceRmContext = createContext<VoiceRmConfig | null>(null);

export function VoiceRmProvider({ children, ...config }: VoiceRmConfig & { children: ReactNode }) {
  return <VoiceRmContext.Provider value={config}>{children}</VoiceRmContext.Provider>;
}

export function useVoiceRmConfig(): VoiceRmConfig {
  const ctx = useContext(VoiceRmContext);
  if (!ctx) throw new Error("useVoiceRmConfig must be inside VoiceRmProvider");
  return ctx;
}

```

10.4 useVoiceSession Hook

src/voice/useVoiceSession.ts

```

// packages/voice-rm-sdk/src/voice/useVoiceSession.ts
import { useState, useCallback, useRef } from 'react';
import { useVoiceRmConfig } from '../provider/VoiceRmProvider';

```

```

export function useVoiceSession({ journey, lang }: { journey: string; lang?: string })
{
  const { apiBase, getToken, lang: ctxLang } = useVoiceRmConfig();
  const [session, setSession] = useState<Session | null>(null);
  const [loading, setLoading] = useState(false);
  const sessionIdRef = useRef<string | null>(null);

  const callEngine = useCallback(async (input: TurnInput) => {
    setLoading(true);
    try {
      const token = await getToken();
      const resp = await fetch(`${apiBase}/api/engine/turn`, {
        method: "POST",
        headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
        body: JSON.stringify({ sessionId: sessionIdRef.current, lang: lang || ctxLang,
input })
      });
      const data = await resp.json();
      if (!data.ok) throw Object.assign(new Error(data.error.message), { code:
data.error.code });
      sessionIdRef.current = data.sessionId;
      setSession(data.session);
      return data.session;
    } finally { setLoading(false); }
  }, [apiBase, getToken, lang, ctxLang]);

  const initSession = useCallback(() => callEngine({ type: "INIT" })), [callEngine];
  const sendUtterance = useCallback((text: string) => callEngine({ type: "TRANSCRIPT",
text })), [callEngine];
  const startJourney = useCallback(() => callEngine({ type: "START_ACTION", action:
journey })), [callEngine, journey];
  const confirm = useCallback((stepUpToken: string, idempotencyKey: string) =>
    callEngine({ type: "CONFIRMATION", confirmed: true, stepUpToken, idempotencyKey })),
[callEngine];
  const cancel = useCallback(() => callEngine({ type: "CANCEL" })), [callEngine];

  return { session, loading, initSession, sendUtterance, startJourney, confirm, cancel
};
}

```

10.5 AG-UI Client (SSE Stream Handler)

src/agui/aguiClient.ts

```

// packages/voice-rm-sdk/src/agui/aguiClient.ts
import { useVoiceRmConfig } from '../provider/VoiceRmProvider';

export interface AguiHandlers {
  onStateUpdate?: (patch: JsonPatchOp[]) => void;
  onToolCall?: (toolName: string, args: unknown) => void;
  onText?: (chunk: string) => void;
  onFinished?: () => void;
  onError?: (err: { message: string; code: string }) => void;
}

export async function runAgent(
  params: RunAgentInput,
  handlers: AguiHandlers,
  apiBase: string,
  getToken: () => string | Promise<string>
) {
  const token = await getToken();

```

```

const resp = await fetch(`${apiBase}/api/agui/${params.agentId}`, {
  method: "POST",
  headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
  Accept: "text/event-stream" },
  body: JSON.stringify(params),
});

// NOTE for React Native: replace fetch with expo/fetch or react-native-sse
// Standard React Native fetch does NOT support streaming response body.

const reader = resp.body!.getReader();
const decoder = new TextDecoder();
let buffer = "";

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  const events = buffer.split("\n\n");
  buffer = events.pop() ?? "";
  for (const raw of events) {
    const typeLine = raw.split("\n").find(l => l.startsWith("event:"));
    const dataLine = raw.split("\n").find(l => l.startsWith("data:"));
    if (!typeLine || !dataLine) continue;
    const type = typeLine.slice(7).trim();
    const data = JSON.parse(dataLine.slice(5).trim());
    if (type === "STATE_DELTA") handlers.onStateUpdate?.(data.delta);
    if (type === "TOOL_CALL_END") handlers.onToolCall?.(data.tool_name, data.args);
    if (type === "TEXT_MESSAGE_CONTENT") handlers.onText?.(data.content);
    if (type === "RUN_FINISHED") handlers.onFinished?.();
    if (type === "RUN_ERROR") handlers.onError?.(data);
  }
}

// JSON Patch apply
export function applyStateDelta<T>(state: T, patch: JsonPatchOp[]): T {
  const next = { ...state } as any;
  for (const op of patch) {
    const key = op.path.replace("/", "");
    if (op.op === "replace" || op.op === "add") next[key] = op.value;
    if (op.op === "remove") delete next[key];
  }
  return next as T;
}

```

10.6 useRageDetect Hook

src/behavior/useRageDetect.ts

```

// packages/voice-rm-sdk/src/behavior/useRageDetect.ts
import { useRef, useCallback } from 'react';

interface RageDetectOptions {
  onFrustrated: (context: { invalidFields: string[]; clickCount: number }) => void;
  clickThreshold?: number; // default 5 clicks within window
  windowMs?: number; // default 3000ms
  invalidThreshold?: number; // default 2 invalid fields
}

export function useRageDetect({ onFrustrated, clickThreshold=5, windowMs=3000,
invalidThreshold=2 }: RageDetectOptions) {

```



```

const clicks = useRef<number[]>([]);
const invalidFields = useRef<Set<string>>(new Set());
const triggered = useRef(false);

const checkFrustration = useCallback(() => {
  if (triggered.current) return;
  const now = Date.now();
  clicks.current = clicks.current.filter(t => now - t < windowMs);
  const rageClicks = clicks.current.length >= clickThreshold;
  const manyErrors = invalidFields.current.size >= invalidThreshold;
  if (rageClicks || manyErrors) {
    triggered.current = true;
    onFrustrated({ invalidFields: [...invalidFields.current], clickCount:
clicks.current.length });
    setTimeout(() => { triggered.current = false; }, 60_000); // reset after 1 min
  }
}, [onFrustrated, clickThreshold, windowMs, invalidThreshold]);

const containerProps = {
  onClick: () => { clicks.current.push(Date.now()); checkFrustration(); }
};

const markInvalidField = useCallback((fieldId: string) => {
  invalidFields.current.add(fieldId);
  checkFrustration();
}, [checkFrustration]);

const dismiss = useCallback(() => {
  triggered.current = false;
  invalidFields.current.clear();
  clicks.current = [];
}, []);

return { containerProps, markInvalidField, dismiss };
}

```

10.7 Android Native Bridge Abstraction

src/bridge/nativeBridge.ts

```

// packages/voice-rm-sdk/src/bridge/nativeBridge.ts
// Abstracts the Android __bankBridge JavascriptInterface
// On web (non-WebView) all methods are no-ops or browser-native implementations

declare global { interface Window { __bankBridge?: BankBridgeInterface;
__VOICE_RM_CONFIG?: VoiceRmConfig; } }

interface BankBridgeInterface {
  getAuthToken(): string;
  navigateTo(screen: string, contextJson: string): void;
  verifyMpin(callbackName: string): void;
  logAnalyticsEvent(name: string, payloadJson: string): void;
}

export const nativeBridge = {
  isAndroidWebView: () => typeof window.__bankBridge !== "undefined",

  getAuthToken: (): Promise<string> => {
    if (window.__bankBridge) return
    Promise.resolve(window.__bankBridge.getAuthToken());
    // Web: use injected config (VoiceRmProvider supplies this)
  }
}

```

```
    return Promise.resolve(window.__VOICE_RM_CONFIG__?.jwt || "");
  },

  navigateTo: (screen: string, context: Record<string, unknown>) => {
    if (window.__bankBridge) window.__bankBridge.navigateTo(screen,
      JSON.stringify(context));
    else window.dispatchEvent(new CustomEvent("voice-rm:navigate", { detail: { screen,
      context } }));
  },

  requestMpin: (): Promise<boolean> => new Promise(resolve => {
    if (window.__bankBridge) {
      const cbName = `__mpinCallback_${Date.now()}`;
      (window as any)[cbName] = (ok: boolean) => { resolve(ok); delete (window as
any)[cbName]; };
      window.__bankBridge.verifyMpin(cbName);
    } else {
      // Web fallback: open MPIN modal (bank implements this)
      window.dispatchEvent(new CustomEvent("voice-rm:request-mpin"));
      // Bank resolves via window.__voiceRm.onMpinResult()
      window.addEventListener("voice-rm:mpin-result", (e: any) => resolve(e.detail.ok),
{ once: true });
    }
  })),

  logEvent: (name: string, payload: Record<string, unknown>) => {
    if (window.__bankBridge) window.__bankBridge.logAnalyticsEvent(name,
      JSON.stringify(payload));
  },
};
```

E

PART E

Bank Integration Guide

§9 (Bank)

Bank Integration — Android & Web

Step-by-step code for the bank's integration team

i Bank Team: This entire section is written for the bank's developers. Silver Suits engineers may skip to Part F. All code below is copy-paste ready.

9.1 Android — VoiceRmActivity

VoiceRmActivity.kt — bank writes this

```
// app/src/main/java/com/bank/voice/VoiceRmActivity.kt
class VoiceRmActivity : AppCompatActivity() {
    private lateinit var webView: WebView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_voice_rm)
        webView = findViewById(R.id.voiceWebView)
        setupWebView()
        injectAndLoad()
    }

    private fun setupWebView() {
        webView.settings.apply {
            javaScriptEnabled = true
            domStorageEnabled = true
            allowFileAccess = false           // SECURITY
            allowContentAccess = false        // SECURITY
            mixedContentMode = WebSettings.MIXED_CONTENT_NEVER_ALLOW
            mediaPlaybackRequiresUserGesture = false // allow TTS auto-play
            setSupportMultipleWindows(false)   // SECURITY
        }
        webView.addJavascriptInterface(VoiceRmBridge(this, webView), "__bankBridge")
        webView.webViewClient = VoiceRmWebViewClient()
        webView.webChromeClient = VoiceRmChromeClient(this)
    }

    private fun injectAndLoad() {
        val config = JSONObject().apply {
            put("jwt", SessionManager.currentJwt)
            put("lang", SessionManager.customerLang) // "hi", "te", etc.
            put("userId", SessionManager.cifNumber)
            put("screenId", intent.getStringExtra("screen_id") ?: "home")
            put("apiBase", BuildConfig.VOICE_RM_API_BASE)
        }
        // Inject BEFORE page load so SDK picks up config immediately
        webView.evaluateJavascript("window.__VOICE_RM_CONFIG__ = ${config};", null)
        webView.loadUrl("${BuildConfig.VOICE_RM_CDN_BASE}/embed/v1/index.html")
    }

    // Handle postMessage from SDK on session end
    override fun onBackPressed() {
        if (webView.canGoBack()) webView.goBack() else super.onBackPressed()
    }
}
```

9.2 Android — VoiceRmBridge (JS Interface)

VoiceRmBridge.kt — bank writes this

```
// VoiceRmBridge.kt
class VoiceRmBridge(private val activity: Activity, private val webView: WebView) {

    // Called by SDK to get a fresh JWT (called before each API request)
    @JavascriptInterface
    fun getAuthToken(): String = SessionManager.refreshAndGetJwt() // blocking ok on
    JS thread

    // Called when SDK wants to navigate to a bank screen
    @JavascriptInterface
    fun navigateTo(screen: String, contextJson: String) {
        activity.runOnUiThread {
            BankRouter.open(activity, screen, JSONObject(contextJson))
        }
    }

    // Called when MPIN is needed for CONFIRMATION step
    // callbackName is a JS function name SDK registered on window
    @JavascriptInterface
    fun verifyMpin(callbackName: String) {
        activity.runOnUiThread {
            MpinActivity.launch(activity) { success: Boolean ->
                val js = "window.$callbackName($success)"
                webView.evaluateJavascript(js, null)
            }
        }
    }

    // Analytics passthrough — forwards SDK events to bank analytics
    @JavascriptInterface
    fun logAnalyticsEvent(name: String, payloadJson: String) {
        BankAnalytics.track(name, JSONObject(payloadJson))
    }
}
```

9.3 Android — Microphone Permission Handler

VoiceRmChromeClient.kt — bank writes this

```
// VoiceRmChromeClient.kt
class VoiceRmChromeClient(private val activity: Activity) : WebChromeClient() {
    var pendingPermRequest: PermissionRequest? = null

    override fun onPermissionRequest(request: PermissionRequest) {
        if (PermissionRequest.RESOURCE_AUDIO_CAPTURE in request.resources) {
            if (ContextCompat.checkSelfPermission(activity,
                Manifest.permission.RECORD_AUDIO)
                == PackageManager.PERMISSION_GRANTED) {
                request.grant(request.resources)
            } else {
                pendingPermRequest = request
                ActivityCompat.requestPermissions(activity,
                    arrayOf(Manifest.permission.RECORD_AUDIO), 101)
            }
        } else {
        }
```

```

        request.deny() // Deny all other permissions
    }
}

// In VoiceRmActivity - handle permission callback
override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<String>,
results: IntArray) {
    super.onRequestPermissionsResult(requestCode, permissions, results)
    if (requestCode == 101) {
        val granted = results.firstOrNull() == PackageManager.PERMISSION_GRANTED
        (webView.webChromeClient as VoiceRmChromeClient).pendingPermRequest?.let { req
->
            if (granted) req.grant(req.resources) else req.deny()
        }
    }
}
}

```

9.4 Android — Security Config

network_security_config.xml + WebViewClient — bank writes this

```

<!-- app/src/main/res/xml/network_security_config.xml -->
<!-- Block all cleartext traffic; allowlist only our CDN and API domains -->
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <base-config cleartextTrafficPermitted="false" />
    <domain-config cleartextTrafficPermitted="false">
        <domain includeSubdomains="true">voice-cdn.bank.in</domain>
        <domain includeSubdomains="true">api.voice-rm.bank.in</domain>
    </domain-config>
</network-security-config>

<!-- AndroidManifest.xml - add these inside <application> -->
<!-- android:networkSecurityConfig="@xml/network_security_config" -->
<!-- android:usesCleartextTraffic="false" -->

<!-- Also add in WebViewClient to prevent navigation to unexpected domains -->
// In VoiceRmWebViewClient:
override fun shouldOverrideUrlLoading(view: WebView, request: WebResourceRequest):
Boolean {
    val allowed = listOf("voice-cdn.bank.in", "api.voice-rm.bank.in")
    return request.url.host !in allowed // block all other domains
}

```

9.5 Android — AndroidManifest & Deep Link Wiring

AndroidManifest.xml additions — bank writes this

```

<!-- AndroidManifest.xml -->
<activity android:name=".voice.VoiceRmActivity"
    android:windowSoftInputMode="adjustResize"
    android:configChanges="orientation|screenSize">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <!-- Deep link: bankapp://voice?screen=imps_transfer -->

```

```

        <data android:scheme="bankapp" android:host="voice" />
    </intent-filter>
</activity>

// Launch from FAB button in any Activity:
fun openVoiceRm(screenId: String = "home") {
    startActivity(Intent(this, VoiceRmActivity::class.java).apply {
        putExtra("screen_id", screenId)
    })
}

```

9.8 Web — NPM SDK Integration

Installation

```

# Install the SDK
npm install @silversuits/voice-rm-sdk

# .env (do not commit actual values)
VITE_VOICE_RM_API_BASE=https://api.voice-rm.bank.in

```

src/App.tsx — bank writes this

```

// src/App.tsx — wrap at root level with VoiceRmProvider
import { VoiceRmProvider } from '@silversuits/voice-rm-sdk';
import { bankAuth } from './auth';

export function App() {
    return (
        <VoiceRmProvider
            apiBase={import.meta.env.VITE_VOICE_RM_API_BASE}
            getToken={() => bankAuth.getValidJwt()} // must return fresh JWT each call
            lang={userPrefs.lang}
            onError={err => monitoring.error("voice-rm", err)}
        >
            <BankAppRoutes />
        </VoiceRmProvider>
    );
}

```

9.9 Web — Mounting on a Form Page

src/pages/ImpsTransferPage.tsx — bank writes this (same pattern for all forms)

```

// src/pages/ImpsTransferPage.tsx
import { AguiPanel, VoiceModal, useVoiceSession, useRageDetect, applyStateDelta }
    from '@silversuits/voice-rm-sdk';

export function ImpsTransferPage() {
    const [form, setForm] = useState({ beneficiary_name:"", account_number:"",
    ifsc_code:"", amount:"" });
    const { session, sendUtterance, initSession } = useVoiceSession({
        journey:"fund_transfer", lang:"hi" });

    const { containerProps, markInvalidField } = useRageDetect({

```

```

onFrustrated: ({ invalidFields, clickCount }) => {
  // Call bank's own backend which then calls POST /api/trigger
  bankApi.post("/internal/voice-trigger", {
    event_type: "rage_click",
    screen_id: "imps_transfer",
    field_context: { invalid_fields: invalidFields, click_count: clickCount }
  });
}
});

return (
  <div {...containerProps}>
    /* AG-UI panel pre-fills the form via SSE – mount above the form */
    <AguiPanel
      agentId="indian_bank_imps_transfer"
      formState={form}
      onStateUpdate={(patch) => setForm(s => applyStateDelta(s, patch))}
    />

    /* Existing bank form – add onInvalid callbacks */
    <ImpsForm
      state={form}
      onFieldChange={({field, val}) => setForm(s => ({...s, [field]: val}))}
      onFieldInvalid={({fieldId}) => markInvalidField(fieldId)}
    />

    /* Voice modal – mic button triggers voice journeys */
    <VoiceModal
      session={session}
      onUtterance={sendUtterance}
      onOpen={initSession}
    />
  </div>
);
}

```

9.10 Web — CSP Headers Update

The bank's web server must allow the Silver Suits API domain in its Content Security Policy. Add these directives:

CSP Header Update — bank ops/DevOps

```

# Nginx / Express – add to existing CSP header
# connect-src: allow Silver Suits API for fetch/XHR/EventSource
Content-Security-Policy:
  default-src 'self';
  connect-src 'self' https://api.voice-rm.bank.in;
  media-src 'self' blob:; # allow TTS audio blob URLs
  script-src 'self' 'nonce-{random}';
  worker-src blob:; # allow audio worklets

```


F

PART F

Infrastructure, DevOps & Observability

§11

Infrastructure & Azure Setup

Resources · Environment config · Secrets · Terraform

11.1 Azure Resources Required

Resource	SKU (Staging / Prod)	Purpose	Region
Azure APIM	Developer / Standard	JWT validation, rate limiting, CORS, TLS, routing	India Central
App Service Plan	S2 / P2v3 (Linux)	Hosts all 4 server services via Docker	India Central
Azure Cache for Redis	C1 / C3 (6.x, SSL)	Session store, rate limit counters, jti blacklist, rule cache	India Central
Azure Key Vault	Standard	All API keys, JWT public keys, TLS certs, mTLS certs	India Central
Azure Application Insights	Pay-per-use	Logs, traces, metrics, dashboards, alerts	India Central
Azure Container Registry	Basic / Standard	Docker image storage for App Service deployments	India Central
Azure Private Endpoint	Standard	Secure tunnel to bank VPC Mumbai — Private Link	India Central → Mumbai
Azure Load Testing	Pay-per-use	V2 load test: 1,000 concurrent sessions	India Central

⚠ ALL resources must be provisioned in India Central (East India as V3 DR failover). RBI data localisation: no customer data or AI calls may leave Indian data centres.

11.2 Environment Variables — Complete Reference

Variable	Source	Example / Notes
AZURE_OPENAI_ENDPOINT	Key Vault secret	https://eastindia.openai.azure.com/ — India endpoint only
AZURE_OPENAI_API_KEY	Key Vault secret	(secret) — never in code or .env
AZURE_OPENAI_DEPLOYMENT	App Service env	gpt-4o — deployment name in Azure OpenAI studio
ELEVENLABS_API_KEY	Key Vault secret	(secret) — India-residency key requested from ElevenLabs
CARTESIA_API_KEY	Key Vault secret	(secret) — India endpoint key
REDIS_URL	Key Vault secret	rediss://voice-rm-prod.redis.cache.windows.net:6380 — SSL required

Variable	Source	Example / Notes
BANK_DB_URI	Key Vault secret	mongodb+srv://sswriter:...@bank-cluster.mongodb.net/ — rotate quarterly
BANK_API_BASE	Key Vault secret	https://coreapi.bank.internal — bank UAT and prod separate
BANK_SERVICE_TOKEN	Key Vault secret	Token Silver Suits uses to call bank APIs (not customer JWT)
BANK_JWKS_URI	App Service env	https://iam.bank.in/.well-known/jwks.json
JWT_AUDIENCE	App Service env	voice-rm-platform — must match aud claim in bank-issued JWTs
JWT_ISSUER	App Service env	https://iam.bank.in — must match iss claim
TRIGGER_SERVICE_TOKEN	Key Vault secret	Token we issue; bank uses this to call POST /api/trigger
MTLS_CERT_PATH	Key Vault cert	Path to client cert for bank Private Link mTLS
MTLS_KEY_PATH	Key Vault secret	Private key for mTLS cert
ALLOWED_ORIGINS	App Service env	https://netbanking.bank.in,https://m.bank.in (comma-sep, NO wildcard)
NODE_ENV	App Service env	production — disables demo routes and verbose errors
LOG_LEVEL	App Service env	warn (prod), info (staging)
APP_VERSION	App Service env	Injected by CI at deploy time (git SHA short)

11.3 CI/CD Pipeline

GitHub Actions CI/CD Pipeline

```
# .github/workflows/deploy.yml
name: Deploy Voice RM
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20" }
      - run: npm ci
      - run: npm run lint
      - run: npx ajv validate -s server/manifests/schema.json -d
        "server/manifests/*.json"
      - run: npm test --workspace=server # unit + scripted flow tests (LLM stubbed)
      - run: npm run typecheck --workspace=packages/voice-rm-sdk

  build:
    needs: test
    if: github.ref == "refs/heads/main"
    steps:
      - uses: actions/checkout@v4
      - uses: azure/docker-login@v1
```

```

    with: { login-server: ${ secrets.ACR_URL }, username: ${ secrets.ACR_USER
}}, password: ${ secrets.ACR_PASS }} }
    - run: |
        docker build -t $ACR_URL/voice-rm:${ github.sha } ./server
        docker push $ACR_URL/voice-rm:${ github.sha }
    - run: npm run build --workspace=packages/voice-rm-sdk
    - run: npm publish --workspace=packages/voice-rm-sdk # to internal registry

deploy-staging:
  needs: build
  environment: staging
  steps:
    - run: az webapp config container set --name voice-rm-staging ...
    - run: npm run smoke-test # /api/health + scripted send_money against staging

deploy-prod:
  needs: deploy-staging
  environment: production # requires manual approval in GitHub Environments
  steps:
    # Blue/green via App Service deployment slots
    - run: az webapp deployment slot swap --name voice-rm-prod --slot staging

```

11.4 Docker Image

server/Dockerfile

```

# server/Dockerfile
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:20-alpine
# Non-root user for security
RUN addgroup -S voicerm && adduser -S voicerm -G voicerm
WORKDIR /app
COPY --from=deps --chown=voicerm:voicerm /app/node_modules ./node_modules
COPY --chown=voicerm:voicerm . .
USER voicerm

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD wget -qO- http://localhost:3000/api/health || exit 1

CMD ["node", "src/index.js"]

```

§12

Observability & Incident Response

Logging · Tracing · Alerts · Runbooks

12.1 Structured Logging (Pino + PII Redaction)

server/src/lib/log.js

```
// server/src/lib/log.js
const pino = require('pino');

// PII patterns to redact before ANY log write
const PII_PATTERNS = [
  { re: /\b\d{10,16}\b/g, mask: "****ACC****" }, // account numbers
  { re: /([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\b)/g, mask: "****UPI****" }, // UPI VPAs (simplified)
  { re: /\b[6-9]\d{9}\b/g, mask: "****PHONE****" }, // Indian mobile numbers
  { re: /\b\d{4}\s\d{4}\s\d{4}\b/g, mask: "****UID****" }, // Aadhaar-like patterns
];

export function piiRedact(text) {
  let safe = text;
  for (const p of PII_PATTERNS) safe = safe.replace(p.re, p.mask);
  return safe;
}

const logger = pino({
  level: process.env.LOG_LEVEL || "info",
  redact: { paths: ["req.headers.authorization", "body.stepUpToken"], censor: "[REDACTED]" },
  transport: process.env.NODE_ENV !== 'production'
    ? { target: "pino-pretty" }
    : { target: "pino-applicationinsights",
      options: { connectionString: process.env.APPINSIGHTS_CONNECTION_STRING } }
});

module.exports = { logger, piiRedact };
```

12.2 Observability Stack

Signal	Tool	Key Metrics	Alert Threshold
Structured logs	Pino → App Insights	turn_latency_ms, intent, outcome, error_code, session_id	Error rate > 2% in 5-min window
Distributed traces	OpenTelemetry → App Insights	STT / LLM / TTS / bank-adapter span durations	p95 total turn > 3s
Redis metrics	Azure Monitor	Hit rate, connected clients, memory %	Memory > 80% or connection error
AG-UI SSE metrics	Custom counter in agui-service	Active SSE connections, run_error rate, run_finished rate	run_error rate > 5%
Bank adapter	Circuit breaker events	Circuit open/close events, timeout rate per endpoint	Any circuit open event
Business metrics	App Insights workbook	Sessions/day, completion rate %, escalation rate, language split	Completion rate < 60%

12.3 Incident Runbooks

Redis Down

1. App Insights auto-alerts within 60 seconds of connection error.

2. Sessions degrade to in-process Map for ≤60 seconds (fail-open). New session creation fails after that with 503.
3. DevOps restarts Azure Redis Cache (auto-recovery ~2 min for non-data-loss restart).
4. If >5 min: set REDIS_URL to DR replica; redeploy via deployment slot swap.

Azure OpenAI Unavailable

5. voice-engine returns "could not understand, please try again" after 1 retry.
6. Trigger service fires assign_human_rm for affected sessions.
7. If >10 min: set maintenance flag in Key Vault → SDK shows "Voice assistant temporarily unavailable."

Bank Core API Down

8. bank-adapter circuit opens after 3 consecutive failures within 30 seconds.
9. Any pending CONFIRMATION returns FAILED state. Idempotency key retained.
10. Customer sees failure message + option to speak with human RM.
11. Alert bank's on-call team. Circuit auto-closes when bank API recovers (30-second reset timer).

G

PART G

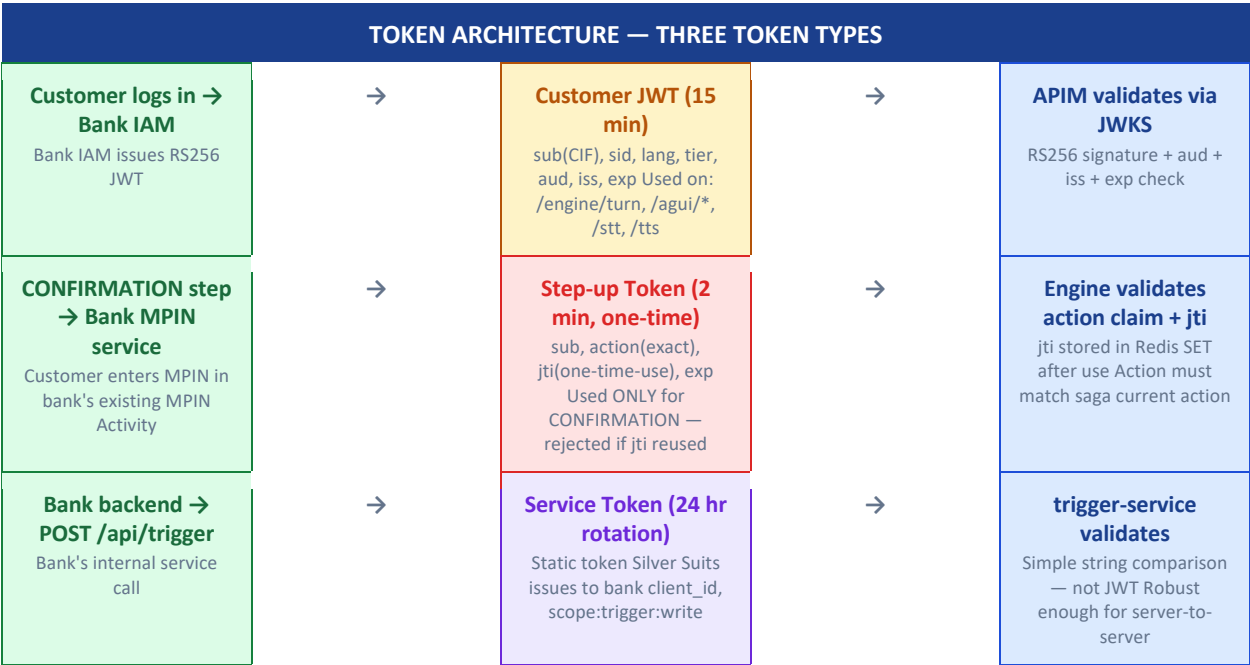
Security & Compliance

§13

Security Controls & Compliance

P0 controls · Auth design · VAPT · Data handling · Token flow

13.1 Token Architecture



13.2 P0 Security Controls — Pre Go-Live Checklist

✗ MANDATORY: Every item below is a hard requirement. No production deployment without all items verified.

Control	Implementation Detail	Verified By
JWT RS256 validation	APIM policy: reject missing/expired/wrong-algorithm token before reaching any service. Gateway middleware double-validates.	QA + VAPT
No wildcard CORS	ALLOWED_ORIGINS env var is a comma-separated list. Express gateway rejects all unlisted origins at preflight.	QA automated test
API keys in Key Vault only	No secrets in .env files, Dockerfiles, or source code. Managed Identity only. Verified by GitHub Advanced Security secret scan on every PR.	CI gate (secret scan)
PII redaction in all logs	piiRedact() applied before every Pino write. Account numbers, VPAs, phone numbers, Aadhaar patterns masked. Log review in UAT by security team.	Security team log audit

Control	Implementation Detail	Verified By
Rate limiting — engine	60 turns/user/hour via APIM policy + Redis rate-limit-redis store. Returns 429 with RATE_LIMITED code.	Load test + QA
Rate limiting — AG-UI	20 agent runs/user/hour. Same Redis store.	Load test + QA
Idempotency on payments	X-Idempotency-Key required on CONFIRMATION and all payment adapter calls. Duplicate key returns 409 DUPLICATE_IDEMPOTENCY within 24h window.	QA scripted test
Step-up token one-time use	jti claim written to Redis SET on first validation. Second use returns 403 INVALID_STEP_UP. TTL matches token expiry.	QA scripted test
Step-up token action binding	action claim in step-up token must exactly match session.action. Prevents token reuse across journeys.	QA scripted test
No audio stored	speech-proxy processes audio in-memory only. Never writes audio data to disk, Redis, or DocumentDB.	Code review + VAPT
mTLS to bank VPC	bank-adapter uses client cert from Key Vault for all bank API calls. Cert rotated every 90 days.	DevOps + bank network team
TLS 1.2+ only	APIM policy blocks TLS < 1.2. App Service min TLS setting enforced.	APIM config + VAPT
Consent gate	First engine turn blocked if consent_accepted claim missing from JWT. Consent logged to DocumentDB.	QA scripted test

13.3 VAPT Scope (Sprint 7–8, Bank-Approved Vendor)

Test Area	What to Test
OWASP API Security Top 10	All /api/* endpoints for injection, auth failure, excessive data exposure, BOLA, BFLA, mass assignment
JWT manipulation	Expired, wrong algorithm (none/HS256), tampered claims, missing aud/iss, wrong audience
Rate limit bypass	IP rotation, header spoofing (X-Forwarded-For), concurrent requests to bypass per-user limits
Android WebView bridge	Enumerate all @JavascriptInterface methods; test for unintended access, parameter injection
SSE stream	Thread_id spoofing (can user A read user B's SSE stream?), event injection
Step-up token attacks	Replay (same jti twice), cross-action reuse (use token for transfer to execute loan)
Bank adapter SSRF	Craft malicious bank API responses to test for server-side request forgery via circuit breaker
PII in logs	Review App Insights logs for any raw account numbers, VPAs, phone numbers, Aadhaar

H

PART H

Testing Strategy

§14

Testing Strategy — Pyramid to Production

Unit · Integration · Contract · E2E · Load · Security

14.1 Test Pyramid

Layer	What is Tested	Tools	CI Gate	Who Owns
Unit	confirmParser, extractSlots mock, applyStateDelta, ruleEngine matchesConditions, piiRedact	Vitest / Node test runner	Every PR — blocks merge	All devs
Integration — Scripted Flows	5 complete voice journeys INIT→DONE with LLM stubbed vianock — happy path + CANCEL + step-up fail	runScriptedFlows.test.js	Every PR — blocks merge	Backend + Conv Eng
Contract	OpenAPI schema validation for all endpoints: /engine/turn, /agui/*, /trigger, /stt, /tts	Schemathesis / Dredd	Weekly scheduled	QA
E2E — WebView	Full Android WebView: rage- click detection → trigger → voice → UPI confirmation	Playwright + Firebase Test Lab device farm	Pre-release tag	QA
Load	1,000 concurrent sessions: engine turns + AG-UI runs mixed	k6 + Azure Load Testing	V2 Sprint 17–18 gate	DevOps + QA
Security — VAPT	Full API security, JWT attacks, Android bridge, SSE, PII logs	Bank-approved vendor + GitHub Adv Security	Pre go-live (Sprint 7– 8)	QA + External vendor

⚠️ Never call real Azure OpenAI in CI tests. All LLM calls must be stubbed with nock. This prevents cost, flakiness, and data policy violations for test data.

14.2 Scripted Flow Test — Complete Example

server/tests/scripted/sendMoney.test.js

```
// server/tests/scripted/sendMoney.test.js
import { describe, it, expect, beforeEach } from 'vitest';
import nock from 'nock';
import { flowRunner } from '../helpers/flowRunner';

beforeEach(() => {
  // Stub Azure OpenAI - returns deterministic slot extraction
  nock("https://eastindia.openai.azure.com")
    .post("/openai/deployments/gpt-4o/chat/completions")
    .reply(200, {
```

```

    choices: [{ message: { role:"assistant", content:
"{'beneficiary': 'Rahul', 'amount': 500}" } } ]
  });
  // Stub bank adapter - successful UPI transfer
 nock(process.env.BANK_API_BASE).post('/payments/upi').reply(200, { txn_ref: 'TXN123',
status: 'SUCCESS' });
  // Stub step-up token verification
  nock(process.env.BANK_API_BASE).post('/auth/verify-step-up').reply(200, { valid: true
});
});

describe('send_money journey - Hindi end-to-end', () => {
  it('completes INIT → TRANSCRIPT → CONFIRM → DONE', async () => {
    const steps = [
      { input: { type:'INIT', lang:'hi' }, expectState: 'IDLE' },
      { input: { type:'START_ACTION', action:'send_money' }, expectState: 'FILL',
        expectPending: { slot: 'beneficiary' } },
      { input: { type:'TRANSCRIPT', text:'Rahul ko 500 bhejo' }, expectState: 'FILL',
        expectPending: { slot: 'amount' } }, // LLM stubbed - partial extract
      { input: { type:'TRANSCRIPT', text:'paanch sau' }, expectState: 'CONFIRM' },
      { input: { type:'CONFIRMATION', confirmed:true, stepUpToken:"mock_token",
        idempotencyKey: crypto.randomUUID() }, expectState: "DONE" },
    ];

    for (const step of steps) {
      const result = await flowRunner.turn(step.input);
      expect(result.session.state).toBe(step.expectState);
      if (step.expectPending?.slot)
        expect(result.session.pending.slot).toBe(step.expectPending.slot);
    }
  });
});

```

14.3 Load Test Specification (k6)

tests/load/loadTest.js

```

// tests/load/loadTest.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration:"5m", target:200 }, // ramp up
    { duration:"10m", target:1000 }, // sustained 1K concurrent
    { duration:"5m", target:0 }, // ramp down
  ],
  thresholds: {
    'http_req_duration{scenario:engine}': ['p(95)<1500'], // p95 < 1.5s
    'http_req_duration{scenario:agui}': ['p(95)<2000'], // AG-UI SSE slower ok
    'http_req_failed': ['rate<0.01'], // < 1% errors
    'checks': ['rate>0.99'],
  },
};

export function engine turn() {
  const jwt = getTestJwt(__VU); // unique JWT per virtual user
  const res = http.post(`${__ENV.BASE_URL}/api/engine/turn`,
    JSON.stringify({ input: { type:"TRANSCRIPT", text:"Send 1000 to Rahul" },
      lang:"hi", sessionId: getOrCreateSession(__VU) }),
    { headers: { Authorization:`Bearer ${jwt}`, "Content-Type":"application/json" },
      tags: { scenario: "engine" } });
  check(res, { "status 200": r => r.status === 200 });
}

```

```
    sleep(1);  
}  
  
export default engine_turn;
```

I

PART I

Roadmap & Delivery

§15

36-Week Development Roadmap

V1 MVP · V2 Production · V3 Intelligence · Team · Go-live gate

15.1 V1 MVP — Weeks 1–8 (Go-Live)

i V1 goal: A bank customer speaks a fund transfer in Hindi on Android → executes against real bank APIs → data persisted → security controls verified.

Sprint	Deliverable	Owner	Acceptance Criteria
1–2	Azure infra via Terraform: APIM, App Service, Redis, Key Vault, App Insights	SS DevOps	Staging URL live; TLS 1.2+; APIM returns 401 without token; secrets in Key Vault
1–2	JWT middleware + CORS allowlist + health endpoint + PII logger	SS Backend	curl no-token → 401; bank test JWT → 200; non-bank origin → rejected
1–2	DocumentDB schema applied to bank UAT cluster; DB write module scaffolded	SS Backend + Bank DBA	sessions + turns + rule_audit collections exist; test write passes
3–4	Redis session store (replaces in-memory Map); 30-min TTL; serialization	SS Backend	2-pod load test; session survives pod restart; TTL confirmed
3–4	voice-rm-sdk v0.1 published to internal npm; VoiceModal + AguiPanel importable	SS Frontend	Bank sample React app imports SDK; TypeScript types work; no peer dep errors
3–4	Android WebView shell — VoiceRmActivity + VoiceRmBridge + mic permission	Bank Mobile	Demo bundle loads; mic granted; navigateTo fires; getAuthToken returns JWT
5–6	Bank adapter v1: accounts, contacts, UPI send against bank UAT API	SS Backend + Bank	send_money executes real UAT transfer; idempotency key blocks duplicate
5–6	POST /api/trigger + rule evaluator + 5 seed rules + rule_audit logging	SS Backend	rage_click → trigger_popup evaluated < 100ms; audit written to DocumentDB
5–6	DocumentDB async write pipeline for sessions + turns (all journeys)	SS Backend + Bank DBA	All turns persisted within 2s; bank DBA can query; no writes block engine response
7–8	Server-side step-up token verification (replaces client mock MPIN 1234)	Bank Auth + SS Backend	CONFIRMATION rejected without valid step-up token; action claim mismatch → 403
7–8	Consent gate enforcement; PII redaction audit in staging logs	SS Backend + QA	Session blocked without consent; no raw account numbers in App Insights logs
7–8	UAT with bank team: all 5 journeys against UAT APIs; VAPT with approved vendor	QA + All	Bank sign-off on all journeys; zero P0 VAPT findings; consent flow approved

15.2 V2 Production — Weeks 9–20

Sprint	Deliverable	Acceptance Criteria
9–10	Push notification actuation (FCM/APNs via bank gateway)	Rule match fires push < 30s; deep link opens correct screen; opt-out respected
11–12	Visual rule-builder UI (bank ops dashboard)	Bank ops edits/disables rules without deployment; all changes audit-logged
11–12	6 additional languages (Gujarati, Kannada, Punjabi, Marathi, Malayalam, Bengali)	All 10 languages pass scripted flow tests; STT + manifests + TTS verified
13–14	Emergency override dashboard (disable any rule in < 60s)	Ops disables rule; takes effect within rule-cache TTL (60s); audit logged
15–16	Analytics dashboard v1: sessions/day, completion rate, escalation rate, language split	Dashboard live; data < 5-min stale; accessible to bank product team
17–18	Load test: 1,000 concurrent sessions against staging	p95 turn latency < 1.5s; zero session loss; no circuit breaker trips under load
19–20	Migrate aguiClient.ts → official @ag-ui/client HttpAgent	All 5 agents pass AG-UI compatibility suite; no regression in E2E tests

15.3 V3 Intelligence — Weeks 21–36

Sprint	Deliverable	Acceptance Criteria
21–24	Post-session LLM analysis worker (async queue consumer)	sentiment_arc, intent_category, qa_score written per session within 5 min of session end
25–28	Churn risk scoring + predictive trigger rules	Rules evaluate churn_risk_score; scores validated against bank historical data (AUC > 0.7)
29–32	Full analytics: conversation replay, multi-journey funnel, cohort analysis	Bank product team replays any session; funnel shows drop-off by step
33–36	Multi-region DR (East India failover) + ISO 27001 audit prep	Failover RTO < 15 min; DR drill passed; audit evidence pack complete

15.4 Team Allocation

Role	FTE	V1 Focus (Weeks 1–8)	V2–V3 Focus
Backend Engineer	2	voice-engine hardening, Redis, trigger API, bank adapter	Rule builder API, analytics worker, @ag-ui/client migration
Frontend / SDK Engineer	1	voice-rm-sdk build, WebView bundle, AguiPanel, SDK publishing	Rule builder UI, analytics dashboard, V2 SDK features
AI / Conversation Engineer	1	Manifest tuning, AG-UI agent prompts, scripted flow eval harness	Post-session LLM pipeline, language expansion (V2)
DevOps	1	Azure Terraform, CI/CD, APIM config, Key Vault, App Insights alerts	DR setup, load test infra, ISO 27001 evidence collection

Role	FTE	V1 Focus (Weeks 1–8)	V2–V3 Focus
QA Engineer	1	Scripted flows, UAT scripts, VAPT coordination, regression suite	Load test execution, security regression, audit evidence
Bank Mobile Dev (bank)	1	VoiceRmActivity, Bridge, mic permission, Android security	Push deep links, native mic polish, React Native path (V2)
Bank Frontend Dev (bank)	1	VoiceRmProvider setup, AguiPanel on 4 form pages, CSP update	Dashboard embed, new journey pages
Bank Backend Dev (bank)	1	JWT/JWKS config, /api/trigger caller, DB schema, UAT core APIs	CRM webhook, push gateway integration, contract tests
Product Manager	1	Sprint planning, bank stakeholder sync, acceptance criteria	Analytics requirements, rule-builder UX, V3 roadmap

15.5 Demo → Production Migration Checklist

Demo Artifact	Location	Production Replacement	Sprint
In-memory Map session store	server/engine/session.js	Azure Redis Cache + serialization + 30-min TTL	3–4
Mock banking REST API	server/data/backend.js	bankAdapter → real bank core APIs with circuit breaker	5–6
Hardcoded MPIN "1234" client-side	client/MpinSheet.jsx	Bank MpinActivity + server-side step-up token verification	7–8
Open CORS cors()	server/index.js	APIM CORS policy + ALLOWED_ORIGINS allowlist	1–2
VITE_API_BASE inconsistency	Multiple client files	Single SDK config: apiBase on VoiceRmProvider	3–4
Custom SSE aguiClient.js	client/src/lib/aguiClient.js	@ag-ui/client HttpAgent (V2 Sprint 19–20)	19–20
Missing /api/trigger	(absent)	trigger-service + rule engine + DocumentDB config collection	5–6
DemoPanel.jsx and demo routes	client/src/	Remove from production bundle via NODE_ENV=production guard	1–2
console.log logging	server/index.js	Pino structured logging + PII redaction + App Insights sink	1–2
No rate limiting	server/index.js	APIM rate-limit policy + Redis rate-limit-redis counters	1–2
No idempotency	engine confirm handler	X-Idempotency-Key required; Redis duplicate check (24h)	5–6
API keys in .env	.env file	Azure Key Vault + Managed Identity; no .env in Docker image	1–2

15.6 V1 Go-Live Readiness Gate

✘ All items must be signed off by Silver Suits engineering lead AND bank IT security before production deployment.

- [] All PO security controls (§13.2) implemented and verified by VAPT vendor.
- [] All 5 scripted voice journeys pass against staging with real bank UAT APIs.
- [] Bank sign-off on data model: sessions, turns, form_events, rule_audit reviewed.
- [] Consent flow reviewed and approved by bank's legal/compliance team.
- [] PII redaction verified: manual App Insights log review shows no raw sensitive data.
- [] Disaster recovery plan documented and primary failure modes tested (§12.3).
- [] Android WebView shell security review passed by bank mobile security team.
- [] Key rotation process documented and first rotation tested.
- [] App Insights dashboards + PagerDuty alerts configured and smoke-tested.
- [] SDK v1.0 published to bank's internal npm registry with changelog and API types.
- [] Bank integration team has completed integration testing on staging.

J

PART J

Complete API Reference

§16

API Contracts & Reference

Endpoints · Request/response · Error codes · Data models · Language codes

16.1 All Endpoints

Method	Endpoint	Auth	Purpose
GET	/api/health	None	Health check — {status:"ok", version}
POST	/api/engine/turn	Customer JWT	Voice turn — main interaction endpoint
POST	/api/agui/:agentId	Customer JWT	AG-UI form assistant (SSE response stream)
POST	/api/stt	Customer JWT	Speech-to-text proxy (ElevenLabs Scribe v1)
POST	/api/tts	Customer JWT	Text-to-speech proxy (Cartesia Sonic-2)
POST	/api/trigger	Service Token	Behavioral trigger rule evaluation + actuation
POST	/api/consent	Customer JWT	Record customer consent for voice features
GET	/api/sessions/:sessionId	Customer JWT (own session only)	Get session state for reconnect

16.2 POST /api/engine/turn — Complete Contract

```
POST /api/engine/turn

// REQUEST
POST /api/engine/turn
Authorization: Bearer {customer_jwt}
X-Idempotency-Key: {uuid} // REQUIRED on CONFIRMATION and any payment input
X-Bank-Session-Id: {bank_sid} // Optional — bank mobile session for cross-system correlation
Content-Type: application/json

{
  "sessionId": "sess_abc123", // Omit on first call (INIT)
  "lang": "hi",
  "input": {
    "type": "TRANSCRIPT", // INIT | START_ACTION | TRANSCRIPT | SELECTION |
                          // CONFIRMATION | CANCEL | SET_LANG
    "text": "Rahul ko 500 bhejo"
    // For CONFIRMATION: { type, confirmed, stepUpToken, idempotencyKey }
    // For START_ACTION: { type, action } e.g. action:"send_money"
    // For SELECTION: { type, optionId } e.g. optionId:"acc_001"
    // For SET_LANG: { type, lang } e.g. lang:"te"
  }
}

// SUCCESS RESPONSE
{
  "ok": true,
```

```

    "sessionId": "sess_abc123",
    "session": {
      "state": "FILL", // IDLE | FILL | DISAMBIGUATE | CHOOSE | CONFIRM |
EXECUTING | DONE | FAILED | CANCELLED
      "action": "send_money",
      "pending": {
        "kind": "free_text", // free_text | single_choice | multi_choice | confirm
        "slot": "amount",
        "prompt": "Kitna paisa bhejna chahte hain?",
        "options": null // array of {id, label} only for choice kinds
      },
      "history": [
        { "role": "assistant", "text": "Rahul ke liye ₹500 bhejna hai, confirm?" }
      ],
      "executing": false
    }
  }

// ERROR RESPONSE
{
  "ok": false,
  "error": { "code": "SESSION_EXPIRED", "message": "Session not found or TTL elapsed" }
}

```

16.3 POST /api/agui/:agentId — AG-UI Contract

POST /api/agui/:agentId — SSE stream

```

POST /api/agui/indian_bankimps_transfer
Authorization: Bearer {customer_jwt}
Accept: text/event-stream
Content-Type: application/json

{
  "thread_id": "uuid-v4", // stable per user session; reuse on reconnect
  "run_id": "uuid-v4", // fresh per request: crypto.randomUUID()
  "state": { "beneficiary_name": "", "account_number": "", "ifsc_code": "", "amount": "" },
  "messages": [ { "id": "msg_1", "role": "user", "content": "Transfer 50000 to Rahul" } ],
  "tools": [],
  "context": [ { "type": "user_profile", "data": { "lang": "hi", "tier": "gold",
"cif": "CIF123" } } ],
  "forwarded_props": { "screen_id": "imps_transfer", "journey": "fund_transfer" }
}

// SSE RESPONSE STREAM (Content-Type: text/event-stream)
event: RUN_STARTED
data: {"thread_id": "...", "run_id": "..."}

event: TOOL_CALL_START
data: {"tool_call_id": "tc_1", "tool_name": "set_field"}

event: TOOL_CALL_ARGS
data: {"tool_call_id": "tc_1", "args": {"field_id": "beneficiary_name", "value": "Rahul
Sharma"}}

event: TOOL_CALL_END
data: {"tool_call_id": "tc_1"}

event: STATE_DELTA
data: {"delta": [{"op": "replace", "path": "/beneficiary_name", "value": "Rahul Sharma"}]}

```

```

event: TOOL_CALL_RESULT
data: {"tool_call_id": "tc_1", "result": {"success": true}}

event: RUN_FINISHED
data: {"thread_id": "...", "run_id": "..."}

```

16.4 POST /api/trigger Contract

POST /api/trigger

```

POST /api/trigger
Authorization: Bearer {service_token} // bank backend or bank frontend calls this
Content-Type: application/json

{
  "event_type": "rage_click",           // rage_click | form_abandon | long_dwell |
  explicit_help
  "user_id": "CIF123456",
  "screen_id": "imps_transfer",
  "field_context": { "invalid_field": "ifsc_code", "attempts": 2 },
  "lang": "hi",
  "device": { "platform": "android", "app_version": "4.2.1" }
}

// MATCH RESPONSE
{
  "matched_rule_id": "rule_frustratedimps",
  "action": "trigger_popup",
  "action_config": {
    "script_id": "imps_assist",
    "agent_id": "indian_bankimps_transfer",
    "visibility": "modal"               // modal | bottom_sheet | inline
  }
}

// NO MATCH
{ "matched_rule_id": null, "action": "none" }

```

16.5 DocumentDB Schema — All Collections

Collection	Key Fields	Written When	Retention
sessions	session_id, user_id (CIF), journey, outcome, lang, platform, turn_count, started_at, ended_at	Session INIT and session end	7 years
turns	session_id, turn_idx, role, text (PII-redacted), intent, slots (JSON), latency_ms, ts	Each engine turn — async write pipeline	7 years
form_events	session_id, field_id, value_hash (SHA-256 — NOT plaintext), valid, agent_id, ts	Each AG-UI set_field tool call	7 years
rule_audit	rule_id, user_id, event_type, screen_id, action, ts, cooldown_expires_at	Every rule evaluation (match or no-match)	7 years

Collection	Key Fields	Written When	Retention
escalations	user_id, rm_id, priority, reason, session_id, ts, resolved_at	assign_human_rm trigger fires	7 years
config_rules	rule JSON (see §8.1), enabled, priority, updated_by, updated_at, version	Rule-builder UI saves (V2)	Indefinite, versioned
analytics	session_id, sentiment_arc[], intent_category, qa_score, churn_risk_score, summary	Post-session LLM worker — 5 min delay (V3)	7 years

✗ NEVER store raw voice audio in any Silver Suits system. NEVER store plaintext account numbers, VPAs, or MPIN. form_events stores SHA-256 hashes of field values only.

16.6 Error Codes Reference

Error Code	HTTP	Meaning	SDK / Client Action
SESSION_EXPIRED	404	Redis TTL elapsed or session not found	Start new session (send INIT turn)
INVALID_STEP_UP	403	Step-up token missing, expired, jti reused, or action mismatch	Prompt user to re-enter MPIN
DUPLICATE_IDEMPOTENCY	409	Same X-Idempotency-Key used twice within 24h	Do NOT retry — show "already submitted"
BANK_API_ERROR	502	Bank core API returned error or timed out	Show failure; offer retry or human RM
CIRCUIT_OPEN	503	Bank adapter circuit breaker is open	Show "service unavailable"; retry after delay
RATE_LIMITED	429	Too many requests from this user	Exponential backoff; show "please wait"
CONSENT_REQUIRED	403	Customer has not accepted voice assistant consent	Show consent screen; retry after consent
AGENT_NOT_FOUND	404	Unknown agentId in POST /api/agui/:agentId	Log bug; fall back to home assistant
INVALID_TOKEN	401	JWT missing, expired, wrong algorithm, or wrong audience	Redirect to bank login to refresh token
MANIFEST_NOT_FOUND	404	Unknown action in START_ACTION	Log bug; return to IDLE with generic help

16.7 Journey & Agent IDs

Journey (Manifest) ID	AG-UI Agent ID	Entry Point	Key Slots
send_money	indian_bankimps_transfer	Voice command or IMPS form page	beneficiary, amount, account_type

Journey (Manifest) ID	AG-UI Agent ID	Entry Point	Key Slots
pay_bill	indian_bank_bill_pay	Bill payment form	biller_id, consumer_number, amount
apply_loan	indian_bank_loan_loan	Loan application form	loan_type, amount, tenure, purpose, income
create_fd	indian_bank_deposit	FD/RD creation form	deposit_type, amount, tenure, nomination
view_statement	indian_bank_txn_history	Transaction history / statement page	date_from, date_to, txn_type
home_assist	indian_bank_home_assistant	Home screen FAB / global	Routes to any of the above journeys

16.8 Supported Languages

Code	Language	V1 or V2	ElevenLabs Model	Cartesia Voice ID
hi	Hindi	V1	hi-IN	indian-hi-female
en	English (Indian)	V1	en-IN	indian-en-female
te	Telugu	V1	te-IN	indian-te-female
ta	Tamil	V1	ta-IN	indian-ta-female
gu	Gujarati	V2 Sprint 11–12	gu-IN	indian-gu-female
kn	Kannada	V2 Sprint 11–12	kn-IN	indian-kn-female
pa	Punjabi	V2 Sprint 11–12	pa-IN	indian-pa-female
mr	Marathi	V2 Sprint 11–12	mr-IN	indian-mr-female
ml	Malayalam	V2 Sprint 11–12	ml-IN	indian-ml-female
bn	Bengali	V2 Sprint 11–12	bn-IN	indian-bn-female

16.9 Glossary

Term	Definition
AG-UI	Agent-User Interaction Protocol — HTTP + SSE standard for streaming AI agent events to frontend UIs. Official docs: docs.ag-ui.com
Manifest / Saga	Declarative JSON definition of a banking journey: slots to collect, prompts, confirmation template, bank adapter call to execute.
Slot	Single piece of information the voice engine needs to complete a journey (e.g. beneficiary name, amount).
Step-up Token	Short-lived JWT (2 min, one-time) issued by the bank's MPIN service after the customer verifies their PIN, authorising a specific payment action.

Term	Definition
Idempotency Key	Client-generated UUID sent with payment requests ensuring duplicate requests (retries) do not cause double payments.
Rage Click	Rapid repeated clicks on a UI element detected by useRageDetect hook, indicating customer frustration. Threshold: 5 clicks in 3 seconds OR 2 invalid fields.
Circuit Breaker	Pattern: after N consecutive bank API failures, stop calling the API for X seconds to allow recovery. Implemented via opossum library.
CIF	Customer Information File number — the bank's unique customer identifier used as user_id throughout the platform.
jti	JWT ID claim — used as a one-time-use identifier for step-up tokens to prevent replay attacks.
PII	Personally Identifiable Information — account numbers, VPAs, phone numbers, Aadhaar-like patterns — redacted before all log writes.
SSE	Server-Sent Events — one-directional HTTP streaming from server to client. Used for AG-UI agent runs. Not WebSocket.
APIM	Azure API Management — ingress layer for TLS termination, JWT validation, rate limiting, and CORS before traffic reaches any service.
Silver Suits AI	The platform owner (us) — builds and operates the AI voice platform and publishes the SDK to partner banks.
Partner Bank	Our client — integrates the voice-rm-sdk into their existing Android app and internet banking website.

Document End — AI Voice RM Platform Technical Implementation Guide v1.0

Silver Suits AI · Confidential · May 2026